

BillSync Frontend AI Prompts

Full 75+24 screen implementation / copy-paste Codex prompts with API wiring

How to use: This document follows the same format as the sample ClipIQ prompt pack, but the branch count is based on BillSync scope, not the sample count. Copy one complete branch prompt into Codex, finish implementation, verify, commit, and open a PR before moving to the next branch.

Scope: Design and implement all BillSync frontend screens: approximately 75 product screens plus 24 fallback/loading/empty/error states, with API connections to the existing backend wherever available.

Source folders: Design source: E:\Shwetal\UI Design\stitch_billsync_legal_platform_ui | Backend/API source: E:\Shwetal\legal-billing-backend

Branch Prompt Index

No.	Branch	Primary Feature Area	Commit Message
A	feat/frontend-bootstrap	React + Tailwind bootstrap, route map for all product pages and fallback states, public/protected layouts, sidebar/topbar shell	chore(frontend): bootstrap BillSync React Tailwind shell
B	feat/design-system	BillSync design system: buttons, forms, status badges, tables, tabs, modals, drawers, toasts, skeletons, empty/error states, progress indicators	feat(ui): add BillSync design system
C	feat/api-client-contracts	API client, request helpers, auth headers, error normalization, pagination helpers, upload helpers, typed service modules for every backend area	feat(api): add frontend API client and service contracts
D	feat/auth-users-permissions	Login, register/invite, forgot/reset password, profile, user management, roles and permissions	feat(auth): implement user access flows
E	feat/fallback-state-system	24 fallback states and reusable fallback gallery: loading, skeleton, empty, no results, offline, session expired, permission denied, unavailable, integration not connected, extension issues, meter failure, AI timeout, payment failure, upload failure, court sync failure	feat(ui): implement fallback state system
F	feat/global-dashboard-common	Role dashboards, lawyer dashboard, admin dashboard, notification center, global search, help/guide center, My Setup Status	feat(dashboard): add global dashboards and setup status
G	feat/clients-api	Client list, create/edit, detail overview, contacts, billing summary	feat(clients): connect client management screens
H	feat/matters-core-api	Matter list, create/edit, matter detail overview, matter team assignments	feat(matters): connect matter core screens
I	feat/matter-timeline-docs-billing-audit	Matter timeline, documents tab, billing tab, audit tab	feat(matters): add timeline documents billing audit tabs

J	feat/tasks-daily-work	My Tasks, task board, task detail, task create/edit, My Work Today	feat(tasks): implement daily task workflows
K	feat/calendar-hearings	Calendar, hearing dashboard, court hearing time entry, calendar not connected state	feat(calendar): add hearing and calendar workflows
L	feat/work-meter-time-capture	Work Meter, work session history, manual time entry, captured work review, submit work for approval	feat(work): connect meter and time capture
M	feat/chrome-extension-setup-status	Chrome extension setup, extension status, extension troubleshooting	feat(extension): add setup status troubleshooting
N	feat/email-research-capture	Gmail capture review, research capture review, capture mapping, conversion to billable/time where available	feat(capture): connect Gmail and research review
O	feat/call-meeting-recorder	Call/meeting recorder, recording library, recording detail with transcript and matter link	feat(recorder): add call meeting recorder screens
P	feat/billables-rates-api	Billables list, billable detail, approval queue, rate cards	feat(billing): connect billables and rates
Q	feat/invoices-api	Invoice list, invoice builder, invoice detail, invoice send/share, invoice templates	feat(invoices): connect invoice workflows
R	feat/payments-portal-reconciliation	Payment dashboard, payment entry, client payment portal, reconciliation, payment failed state	feat(payments): add payment portal and reconciliation
S	feat/finance-reports-audit	Finance dashboard, reports, audit logs, AR/WIP/revenue/utilization views	feat(finance): connect reports and audit screens
T	feat/gst-tds-tax	GST dashboard/settings, TDS dashboard/settings, tax summaries and not-configured states	feat(tax): add GST and TDS screens
U	feat/hr-people	HR dashboard, team directory, employee profile, workload/attendance overview	feat(hr): add people and workload screens
V	feat/payroll	Payroll runs, payroll detail, compensation placeholders, payroll not configured state	feat(payload): add payroll workflow shell
W	feat/storage-documents	Storage library, upload file, document viewer, storage settings, matter document links	feat(storage): connect document storage screens
X	feat/ai-assistant-core	AI assistant, AI email writer, AI research assistant	feat(ai): connect AI assistant core
Y	feat/ai-documents-rag	Document summary, document creation, RAG chatbot, matter document Q&A shell	feat(ai): add document summary creation and RAG screens
Z	feat/communications-whatsapp-sms	WhatsApp inbox/templates, SMS center/templates, communication logs	feat(comms): add WhatsApp SMS communication screens
AA	feat/court-daily-sync	Court daily sync dashboard, court case match, verdict detail, court sync settings	feat(court): add court daily sync screens
AB	feat/zoho-integration-api	Zoho connect/status/sync, WorkDrive link, Zoho sync logs	feat(integrations): connect Zoho workflows
AC	feat/aws-google-	AWS/Google integration setup, cloud	feat(integrations): add

	integrations	storage provider status, integration not connected states	AWS and Google provider screens
AD	feat/settings-admin-controls	Settings, firm setup, permissions matrix refinements, invoice/tax/storage/notification defaults	feat(settings): add admin settings and controls
AE	style/responsive-accessibility-copy-polish	Responsive polish, accessibility, keyboard/focus, contrast, text overflow, reduced motion, lawyer-friendly copy pass	style(ui): polish responsiveness accessibility and copy
AF	test/e2e-demo-hardening	End-to-end demo QA across all product screens and fallback states, smoke tests, seeded-data assumptions, final backend-gap report	test(frontend): harden full product demo flows

A-- feat/frontend-bootstrap

AI Prompt

Prompt from product plan, shortened:

You are a senior frontend engineer building BillSync Legal, a React + Tailwind legal-tech SaaS platform. Use the Stitch UI design source at E:\Shwetal\UI Design\stitch_billsync_legal_platform_ui and the backend/API source at E:\Shwetal\legal-billing-backend. The implementation must cover the complete BillSync frontend scope: roughly 75 product screens plus 24 fallback, loading, empty, and error states. The frontend must connect to real backend APIs where they exist, create clean frontend service adapters where backend APIs are missing, and document every backend gap clearly. The UI is for lawyers and law firm operators who are not technical, so never expose raw technical words to users such as API, backend, payload, token, 401, CORS, endpoint, database, stack trace, or server exception. Translate all failures into calm, plain-language guidance. Use the BillSync palette: background #F8F5EF, panels #FFFFFF, navigation #0B1F3A, primary #123B63, accent gold #C9A227, text #1F2933, secondary text #6B7280, border #E5E7EB, success #15803D, warning #D97706, error #B91C1C. Use subtle purposeful animation only: login spinner, live work-meter clock, pulsing recording dot, extension connected check, recorder waveform, upload progress, AI shimmer, payment spinner, and sync progress.

Branch to create/work on:

feat/frontend-bootstrap

Screens/states/features to be added:

React + Tailwind bootstrap, route map for all product pages and fallback states, public/protected layouts, sidebar/topbar shell

Design reference:

All Stitch navigation and global app shell screens

Backend/API reference:

Inspect E:\Shwetal\legal-billing-backend for route inventory, models, validators, controllers, tests, and docs related to this branch.

How to design:

Create the new app foundation. Add placeholders for all 75 product screens and 24 fallback states so scope is visible from day one.

API connection direction:

Create or update frontend service functions for each real backend route used by this branch. Normalize responses into UI-safe models. If a backend route is missing, add a clearly named adapter and document the backend gap in the final response and PR.

Branch-specific senior-engineer direction:

Favor simple, composable implementation over clever abstractions. Keep UI states explicit and reusable. Connect real backend data first; use honest not-configured states for unavailable MVP expansion APIs.

Acceptance criteria:

App runs; every planned route renders a placeholder; sidebar and mobile nav work; routes are grouped by role and module.

Implementation guardrails:

- Start by inspecting relevant Stitch screens in the design folder.
- Start by inspecting relevant backend route files, controllers, validators, models, tests, and docs.
- Create or update API client functions before wiring pages.
- Connect every available screen action to backend behavior where available.
- Where backend APIs are missing, add a service adapter with a clear TODO/backend-gap note and show honest not-connected/not-configured UI.
- Include loading, empty, error, offline, permission, retry, success, and validation states for every data surface.
- Keep lawyer-facing copy non-technical, calm, and action-oriented.
- Do not modify backend unless the branch explicitly asks for a tiny compatibility fix.
- Verify desktop, tablet, and 360px mobile width.

Verification required before marking this branch complete:

- Run npm install only if dependencies are missing.
- Run npm run dev and manually verify the branch route(s).
- Run npm run build before final response.
- Check at least one API success path and one API failure/fallback path.
- Check at least one empty state.
- Check no horizontal overflow at 360px.
- Confirm user-facing copy contains no technical backend jargon.

Commit message:

chore(frontend): bootstrap Billsync React Tailwind shell

PR template before merging:

Summary

-

Screens / States Implemented

- []

API Mapping

- APIs connected:
 - `GET /api/...`
 - `POST /api/...`
- Frontend adapters added:
 - None / list adapters
- Backend gaps found:
 - None / list gaps

UX States Covered

- [] Loading
- [] Empty
- [] Error
- [] Offline
- [] Permission denied
- [] Retry action
- [] Validation
- [] Success toast

Verification

- [] App builds
- [] Key routes render
- [] Forms validate
- [] API success path checked
- [] API failure path checked
- [] Responsive check completed

Notes for Tester

-

Final response format after implementation:

1. Implementation summary
2. Screens/routes implemented
3. API functions connected
4. Fallback states covered
5. Files changed
6. Verification performed with exact commands
7. Known backend gaps
8. Suggested commit message and PR title

B-- feat/design-system

AI Prompt

Prompt from product plan, shortened:

You are a senior frontend engineer building BillSync Legal, a React + Tailwind legal-tech SaaS platform. Use the Stitch UI design source at E:\Shwetal\UI Design\stitch_billsync_legal_platform_ui and the backend/API source at E:\Shwetal\legal-billing-backend. The implementation must cover the complete BillSync frontend scope: roughly 75 product screens plus 24 fallback, loading, empty, and error states. The frontend must connect to real backend APIs where they exist, create clean frontend service adapters where backend APIs are missing, and document every backend gap clearly. The UI is for lawyers and law firm operators who are not technical, so never expose raw technical words to users such as API, backend, payload, token, 401, CORS, endpoint, database, stack trace, or server exception. Translate all failures into calm, plain-language guidance. Use the BillSync palette: background #F8F5EF, panels #FFFFFF, navigation #0B1F3A, primary #123B63, accent gold #C9A227, text #1F2933, secondary text #6B7280, border #E5E7EB, success #15803D, warning #D97706, error #B91C1C. Use subtle purposeful animation only: login spinner, live work-meter clock, pulsing recording dot, extension connected check, recorder waveform, upload progress, AI shimmer, payment spinner, and sync progress.

Branch to create/work on:

feat/design-system

Screens/states/features to be added:

BillSync design system: buttons, forms, status badges, tables, tabs, modals, drawers, toasts, skeletons, empty/error states, progress indicators

Design reference:

Global Stitch component patterns and fallback suite

Backend/API reference:

Inspect E:\Shwetal\legal-billing-backend for route inventory, models, validators, controllers, tests, and docs related to this branch.

How to design:

Extract reusable components before feature work. Keep components simple, accessible, and responsive.

API connection direction:

Create or update frontend service functions for each real backend route used by this branch. Normalize responses into UI-safe models. If a backend route is missing, add a clearly named adapter and document the backend gap in the final response and PR.

Branch-specific senior-engineer direction:

Favor simple, composable implementation over clever abstractions. Keep UI states explicit and reusable. Connect real backend data first; use honest not-configured states for unavailable MVP expansion APIs.

Acceptance criteria:

Component gallery renders all primitives; states support loading/disabled/focus; palette is centralized.

Implementation guardrails:

- Start by inspecting relevant Stitch screens in the design folder.
- Start by inspecting relevant backend route files, controllers, validators, models, tests, and docs.
- Create or update API client functions before wiring pages.
- Connect every available screen action to backend behavior where available.
- Where backend APIs are missing, add a service adapter with a clear TODO/backend-gap note and show honest not-connected/not-configured UI.

- Include loading, empty, error, offline, permission, retry, success, and validation states for every data surface.
- Keep lawyer-facing copy non-technical, calm, and action-oriented.
- Do not modify backend unless the branch explicitly asks for a tiny compatibility fix.
- Verify desktop, tablet, and 360px mobile width.

Verification required before marking this branch complete:

- Run npm install only if dependencies are missing.
- Run npm run dev and manually verify the branch route(s).
- Run npm run build before final response.
- Check at least one API success path and one API failure/fallback path.
- Check at least one empty state.
- Check no horizontal overflow at 360px.
- Confirm user-facing copy contains no technical backend jargon.

Commit message:

feat(ui): add BillSync design system

PR template before merging:

Summary

-

Screens / States Implemented

- []

API Mapping

- APIs connected:
 - `GET /api/...`
 - `POST /api/...`
- Frontend adapters added:
 - None / list adapters
- Backend gaps found:
 - None / list gaps

UX States Covered

- [] Loading
- [] Empty
- [] Error
- [] Offline
- [] Permission denied
- [] Retry action
- [] Validation
- [] Success toast

Verification

- [] App builds

- [] Key routes render
- [] Forms validate
- [] API success path checked
- [] API failure path checked
- [] Responsive check completed

Notes for Tester

-

Final response format after implementation:

1. Implementation summary
2. Screens/routes implemented
3. API functions connected
4. Fallback states covered
5. Files changed
6. Verification performed with exact commands
7. Known backend gaps
8. Suggested commit message and PR title

C-- feat/api-client-contracts

AI Prompt

Prompt from product plan, shortened:

You are a senior frontend engineer building BillSync Legal, a React + Tailwind legal-tech SaaS platform. Use the Stitch UI design source at E:\Shwetal\UI Design\stitch_billsync_legal_platform_ui and the backend/API source at E:\Shwetal\legal-billing-backend. The implementation must cover the complete BillSync frontend scope: roughly 75 product screens plus 24 fallback, loading, empty, and error states. The frontend must connect to real backend APIs where they exist, create clean frontend service adapters where backend APIs are missing, and document every backend gap clearly. The UI is for lawyers and law firm operators who are not technical, so never expose raw technical words to users such as API, backend, payload, token, 401, CORS, endpoint, database, stack trace, or server exception. Translate all failures into calm, plain-language guidance. Use the BillSync palette: background #F8F5EF, panels #FFFFFF, navigation #0B1F3A, primary #123B63, accent gold #C9A227, text #1F2933, secondary text #6B7280, border #E5E7EB, success #15803D, warning #D97706, error #B91C1C. Use subtle purposeful animation only: login spinner, live work-meter clock, pulsing recording dot, extension connected check, recorder waveform, upload progress, AI shimmer, payment spinner, and sync progress.

Branch to create/work on:

feat/api-client-contracts

Screens/states/features to be added:

API client, request helpers, auth headers, error normalization, pagination helpers, upload helpers, typed service modules for every backend area

Design reference:

No UI image; backend-driven branch

Backend/API reference:

Inspect E:\Shweta\legal-billing-backend for route inventory, models, validators, controllers, tests, and docs related to this branch.

How to design:

Read backend routes and docs, then create a frontend API map covering auth, users, clients, cases, activities, work sessions, time entries, email entries, billables, rates, invoices, payments, reports, analytics, AI, and integrations.

API connection direction:

Create or update frontend service functions for each real backend route used by this branch. Normalize responses into UI-safe models. If a backend route is missing, add a clearly named adapter and document the backend gap in the final response and PR.

Branch-specific senior-engineer direction:

Favor simple, composable implementation over clever abstractions. Keep UI states explicit and reusable. Connect real backend data first; use honest not-configured states for unavailable MVP expansion APIs.

Acceptance criteria:

API modules exist for available backend resources; normalized errors never leak raw technical text into UI.

Implementation guardrails:

- Start by inspecting relevant Stitch screens in the design folder.
- Start by inspecting relevant backend route files, controllers, validators, models, tests, and docs.
- Create or update API client functions before wiring pages.
- Connect every available screen action to backend behavior where available.
- Where backend APIs are missing, add a service adapter with a clear TODO/backend-gap note and show honest not-connected/not-configured UI.
- Include loading, empty, error, offline, permission, retry, success, and validation states for every data surface.
- Keep lawyer-facing copy non-technical, calm, and action-oriented.
- Do not modify backend unless the branch explicitly asks for a tiny compatibility fix.
- Verify desktop, tablet, and 360px mobile width.

Verification required before marking this branch complete:

- Run npm install only if dependencies are missing.
- Run npm run dev and manually verify the branch route(s).
- Run npm run build before final response.
- Check at least one API success path and one API failure/fallback path.
- Check at least one empty state.
- Check no horizontal overflow at 360px.
- Confirm user-facing copy contains no technical backend jargon.

Commit message:

feat(api): add frontend API client and service contracts

PR template before merging:

Summary

-

Screens / States Implemented

- []

API Mapping

- APIs connected:

- `GET /api/...`

- `POST /api/...`

- Frontend adapters added:

- None / list adapters

- Backend gaps found:

- None / list gaps

UX States Covered

- [] Loading

- [] Empty

- [] Error

- [] Offline

- [] Permission denied

- [] Retry action

- [] Validation

- [] Success toast

Verification

- [] App builds

- [] Key routes render

- [] Forms validate

- [] API success path checked

- [] API failure path checked

- [] Responsive check completed

Notes for Tester

-

Final response format after implementation:

1. Implementation summary
2. Screens/routes implemented
3. API functions connected
4. Fallback states covered
5. Files changed
6. Verification performed with exact commands
7. Known backend gaps
8. Suggested commit message and PR title

D-- feat/auth-users-permissions

AI Prompt

Prompt from product plan, shortened:

You are a senior frontend engineer building BillSync Legal, a React + Tailwind legal-tech SaaS platform. Use the Stitch UI design source at E:\Shweta\UI Design\stitch_billsync_legal_platform_ui and the backend/API source at E:\Shweta\legal-billing-backend. The implementation must cover the complete BillSync frontend scope: roughly 75 product screens plus 24 fallback, loading, empty, and error states. The frontend must connect to real backend APIs where they exist, create clean frontend service adapters where backend APIs are missing, and document every backend gap clearly. The UI is for lawyers and law firm operators who are not technical, so never expose raw technical words to users such as API, backend, payload, token, 401, CORS, endpoint, database, stack trace, or server exception. Translate all failures into calm, plain-language guidance. Use the BillSync palette: background #F8F5EF, panels #FFFFFF, navigation #0B1F3A, primary #123B63, accent gold #C9A227, text #1F2933, secondary text #6B7280, border #E5E7EB, success #15803D, warning #D97706, error #B91C1C. Use subtle purposeful animation only: login spinner, live work-meter clock, pulsing recording dot, extension connected check, recorder waveform, upload progress, AI shimmer, payment spinner, and sync progress.

Branch to create/work on:

feat/auth-users-permissions

Screens/states/features to be added:

Login, register/invite, forgot/reset password, profile, user management, roles and permissions

Design reference:

Auth and user screens

Backend/API reference:

Inspect E:\Shweta\legal-billing-backend for route inventory, models, validators, controllers, tests, and docs related to this branch.

How to design:

Wire auth and user APIs. Protect routes by role and show permission-denied UI where needed.

API connection direction:

Create or update frontend service functions for each real backend route used by this branch. Normalize responses into UI-safe models. If a backend route is missing, add a clearly named adapter and document the backend gap in the final response and PR.

Branch-specific senior-engineer direction:

Favor simple, composable implementation over clever abstractions. Keep UI states explicit and reusable. Connect real backend data first; use honest not-configured states for unavailable MVP expansion APIs.

Acceptance criteria:

Login/logout works; current user loads; user list/profile screens connect; role gates work.

Implementation guardrails:

- Start by inspecting relevant Stitch screens in the design folder.
- Start by inspecting relevant backend route files, controllers, validators, models, tests, and docs.
- Create or update API client functions before wiring pages.
- Connect every available screen action to backend behavior where available.

- Where backend APIs are missing, add a service adapter with a clear TODO/backend-gap note and show honest not-connected/not-configured UI.
- Include loading, empty, error, offline, permission, retry, success, and validation states for every data surface.
- Keep lawyer-facing copy non-technical, calm, and action-oriented.
- Do not modify backend unless the branch explicitly asks for a tiny compatibility fix.
- Verify desktop, tablet, and 360px mobile width.

Verification required before marking this branch complete:

- Run npm install only if dependencies are missing.
- Run npm run dev and manually verify the branch route(s).
- Run npm run build before final response.
- Check at least one API success path and one API failure/fallback path.
- Check at least one empty state.
- Check no horizontal overflow at 360px.
- Confirm user-facing copy contains no technical backend jargon.

Commit message:

feat(auth): implement user access flows

PR template before merging:

Summary

-

Screens / States Implemented

- []

API Mapping

- APIs connected:
 - `GET /api/...`
 - `POST /api/...`
- Frontend adapters added:
 - None / list adapters
- Backend gaps found:
 - None / list gaps

UX States Covered

- [] Loading
- [] Empty
- [] Error
- [] Offline
- [] Permission denied
- [] Retry action
- [] Validation
- [] Success toast

Verification

- [] App builds
- [] Key routes render
- [] Forms validate
- [] API success path checked
- [] API failure path checked
- [] Responsive check completed

Notes for Tester

-

Final response format after implementation:

1. Implementation summary
2. Screens/routes implemented
3. API functions connected
4. Fallback states covered
5. Files changed
6. Verification performed with exact commands
7. Known backend gaps
8. Suggested commit message and PR title

E-- feat/fallback-state-system

AI Prompt

Prompt from product plan, shortened:

You are a senior frontend engineer building BillSync Legal, a React + Tailwind legal-tech SaaS platform. Use the Stitch UI design source at E:\Shwetal\UI Design\stitch_billsync_legal_platform_ui and the backend/API source at E:\Shwetal\legal-billing-backend. The implementation must cover the complete BillSync frontend scope: roughly 75 product screens plus 24 fallback, loading, empty, and error states. The frontend must connect to real backend APIs where they exist, create clean frontend service adapters where backend APIs are missing, and document every backend gap clearly. The UI is for lawyers and law firm operators who are not technical, so never expose raw technical words to users such as API, backend, payload, token, 401, CORS, endpoint, database, stack trace, or server exception. Translate all failures into calm, plain-language guidance. Use the BillSync palette: background #F8F5EF, panels #FFFFFF, navigation #0B1F3A, primary #123B63, accent gold #C9A227, text #1F2933, secondary text #6B7280, border #E5E7EB, success #15803D, warning #D97706, error #B91C1C. Use subtle purposeful animation only: login spinner, live work-meter clock, pulsing recording dot, extension connected check, recorder waveform, upload progress, AI shimmer, payment spinner, and sync progress.

Branch to create/work on:

feat/fallback-state-system

Screens/states/features to be added:

24 fallback states and reusable fallback gallery: loading, skeleton, empty, no results, offline, session expired,

permission denied, unavailable, integration not connected, extension issues, meter failure, AI timeout, payment failure, upload failure, court sync failure

Design reference:

Fallback suite from Stitch

Backend/API reference:

Inspect E:\Shwetal\legal-billing-backend for route inventory, models, validators, controllers, tests, and docs related to this branch.

How to design:

Implement reusable state components and apply patterns to existing placeholders.

API connection direction:

Create or update frontend service functions for each real backend route used by this branch. Normalize responses into UI-safe models. If a backend route is missing, add a clearly named adapter and document the backend gap in the final response and PR.

Branch-specific senior-engineer direction:

Favor simple, composable implementation over clever abstractions. Keep UI states explicit and reusable. Connect real backend data first; use honest not-configured states for unavailable MVP expansion APIs.

Acceptance criteria:

Fallback gallery includes all 24 states with plain-language copy and retry/reconnect actions.

Implementation guardrails:

- Start by inspecting relevant Stitch screens in the design folder.
- Start by inspecting relevant backend route files, controllers, validators, models, tests, and docs.
- Create or update API client functions before wiring pages.
- Connect every available screen action to backend behavior where available.
- Where backend APIs are missing, add a service adapter with a clear TODO/backend-gap note and show honest not-connected/not-configured UI.
- Include loading, empty, error, offline, permission, retry, success, and validation states for every data surface.
- Keep lawyer-facing copy non-technical, calm, and action-oriented.
- Do not modify backend unless the branch explicitly asks for a tiny compatibility fix.
- Verify desktop, tablet, and 360px mobile width.

Verification required before marking this branch complete:

- Run npm install only if dependencies are missing.
- Run npm run dev and manually verify the branch route(s).
- Run npm run build before final response.
- Check at least one API success path and one API failure/fallback path.
- Check at least one empty state.
- Check no horizontal overflow at 360px.
- Confirm user-facing copy contains no technical backend jargon.

Commit message:

feat(ui): implement fallback state system

PR template before merging:

Summary

-

Screens / States Implemented

- []

API Mapping

- APIs connected:

- `GET /api/...`

- `POST /api/...`

- Frontend adapters added:

- None / list adapters

- Backend gaps found:

- None / list gaps

UX States Covered

- [] Loading

- [] Empty

- [] Error

- [] Offline

- [] Permission denied

- [] Retry action

- [] Validation

- [] Success toast

Verification

- [] App builds

- [] Key routes render

- [] Forms validate

- [] API success path checked

- [] API failure path checked

- [] Responsive check completed

Notes for Tester

-

Final response format after implementation:

1. Implementation summary
2. Screens/routes implemented
3. API functions connected
4. Fallback states covered
5. Files changed
6. Verification performed with exact commands
7. Known backend gaps
8. Suggested commit message and PR title

F-- feat/global-dashboard-common

AI Prompt

Prompt from product plan, shortened:

You are a senior frontend engineer building BillSync Legal, a React + Tailwind legal-tech SaaS platform. Use the Stitch UI design source at E:\Shwetal\UI Design\stitch_billsync_legal_platform_ui and the backend/API source at E:\Shwetal\legal-billing-backend. The implementation must cover the complete BillSync frontend scope: roughly 75 product screens plus 24 fallback, loading, empty, and error states. The frontend must connect to real backend APIs where they exist, create clean frontend service adapters where backend APIs are missing, and document every backend gap clearly. The UI is for lawyers and law firm operators who are not technical, so never expose raw technical words to users such as API, backend, payload, token, 401, CORS, endpoint, database, stack trace, or server exception. Translate all failures into calm, plain-language guidance. Use the BillSync palette: background #F8F5EF, panels #FFFFFF, navigation #0B1F3A, primary #123B63, accent gold #C9A227, text #1F2933, secondary text #6B7280, border #E5E7EB, success #15803D, warning #D97706, error #B91C1C. Use subtle purposeful animation only: login spinner, live work-meter clock, pulsing recording dot, extension connected check, recorder waveform, upload progress, AI shimmer, payment spinner, and sync progress.

Branch to create/work on:

feat/global-dashboard-common

Screens/states/features to be added:

Role dashboards, lawyer dashboard, admin dashboard, notification center, global search, help/guide center, My Setup Status

Design reference:

Dashboard/common screens

Backend/API reference:

Inspect E:\Shwetal\legal-billing-backend for route inventory, models, validators, controllers, tests, and docs related to this branch.

How to design:

Build the common daily workspace and setup confidence layer.

API connection direction:

Create or update frontend service functions for each real backend route used by this branch. Normalize responses into UI-safe models. If a backend route is missing, add a clearly named adapter and document the backend gap in the final response and PR.

Branch-specific senior-engineer direction:

Favor simple, composable implementation over clever abstractions. Keep UI states explicit and reusable. Connect real backend data first; use honest not-configured states for unavailable MVP expansion APIs.

Acceptance criteria:

Dashboards render with real/adapted data; setup cards show working/needs-attention states.

Implementation guardrails:

- Start by inspecting relevant Stitch screens in the design folder.
- Start by inspecting relevant backend route files, controllers, validators, models, tests, and docs.
- Create or update API client functions before wiring pages.

- Connect every available screen action to backend behavior where available.
- Where backend APIs are missing, add a service adapter with a clear TODO/backend-gap note and show honest not-connected/not-configured UI.
- Include loading, empty, error, offline, permission, retry, success, and validation states for every data surface.
- Keep lawyer-facing copy non-technical, calm, and action-oriented.
- Do not modify backend unless the branch explicitly asks for a tiny compatibility fix.
- Verify desktop, tablet, and 360px mobile width.

Verification required before marking this branch complete:

- Run npm install only if dependencies are missing.
- Run npm run dev and manually verify the branch route(s).
- Run npm run build before final response.
- Check at least one API success path and one API failure/fallback path.
- Check at least one empty state.
- Check no horizontal overflow at 360px.
- Confirm user-facing copy contains no technical backend jargon.

Commit message:

feat(dashboard): add global dashboards and setup status

PR template before merging:

Summary

-

Screens / States Implemented

- []

API Mapping

- APIs connected:
 - `GET /api/...`
 - `POST /api/...`
- Frontend adapters added:
 - None / list adapters
- Backend gaps found:
 - None / list gaps

UX States Covered

- [] Loading
- [] Empty
- [] Error
- [] Offline
- [] Permission denied
- [] Retry action
- [] Validation
- [] Success toast

Verification

- [] App builds
- [] Key routes render
- [] Forms validate
- [] API success path checked
- [] API failure path checked
- [] Responsive check completed

Notes for Tester

-

Final response format after implementation:

1. Implementation summary
2. Screens/routes implemented
3. API functions connected
4. Fallback states covered
5. Files changed
6. Verification performed with exact commands
7. Known backend gaps
8. Suggested commit message and PR title

G-- feat/clients-api

AI Prompt

Prompt from product plan, shortened:

You are a senior frontend engineer building BillSync Legal, a React + Tailwind legal-tech SaaS platform. Use the Stitch UI design source at E:\Shwetal\UI Design\stitch_billsync_legal_platform_ui and the backend/API source at E:\Shwetal\legal-billing-backend. The implementation must cover the complete BillSync frontend scope: roughly 75 product screens plus 24 fallback, loading, empty, and error states. The frontend must connect to real backend APIs where they exist, create clean frontend service adapters where backend APIs are missing, and document every backend gap clearly. The UI is for lawyers and law firm operators who are not technical, so never expose raw technical words to users such as API, backend, payload, token, 401, CORS, endpoint, database, stack trace, or server exception. Translate all failures into calm, plain-language guidance. Use the BillSync palette: background #F8F5EF, panels #FFFFFF, navigation #0B1F3A, primary #123B63, accent gold #C9A227, text #1F2933, secondary text #6B7280, border #E5E7EB, success #15803D, warning #D97706, error #B91C1C. Use subtle purposeful animation only: login spinner, live work-meter clock, pulsing recording dot, extension connected check, recorder waveform, upload progress, AI shimmer, payment spinner, and sync progress.

Branch to create/work on:

feat/clients-api

Screens/states/features to be added:

Client list, create/edit, detail overview, contacts, billing summary

Design reference:

Client screens

Backend/API reference:

Inspect E:\Shwetal\legal-billing-backend for route inventory, models, validators, controllers, tests, and docs related to this branch.

How to design:

Wire client APIs plus invoices/payments summaries where available.

API connection direction:

Create or update frontend service functions for each real backend route used by this branch. Normalize responses into UI-safe models. If a backend route is missing, add a clearly named adapter and document the backend gap in the final response and PR.

Branch-specific senior-engineer direction:

Favor simple, composable implementation over clever abstractions. Keep UI states explicit and reusable. Connect real backend data first; use honest not-configured states for unavailable MVP expansion APIs.

Acceptance criteria:

Client CRUD works; contacts and billing summary render; empty/error states exist.

Implementation guardrails:

- Start by inspecting relevant Stitch screens in the design folder.
- Start by inspecting relevant backend route files, controllers, validators, models, tests, and docs.
- Create or update API client functions before wiring pages.
- Connect every available screen action to backend behavior where available.
- Where backend APIs are missing, add a service adapter with a clear TODO/backend-gap note and show honest not-connected/not-configured UI.
- Include loading, empty, error, offline, permission, retry, success, and validation states for every data surface.
- Keep lawyer-facing copy non-technical, calm, and action-oriented.
- Do not modify backend unless the branch explicitly asks for a tiny compatibility fix.
- Verify desktop, tablet, and 360px mobile width.

Verification required before marking this branch complete:

- Run npm install only if dependencies are missing.
- Run npm run dev and manually verify the branch route(s).
- Run npm run build before final response.
- Check at least one API success path and one API failure/fallback path.
- Check at least one empty state.
- Check no horizontal overflow at 360px.
- Confirm user-facing copy contains no technical backend jargon.

Commit message:

feat(clients): connect client management screens

PR template before merging:

Summary

-

Screens / States Implemented

- []

API Mapping

- APIs connected:

- `GET /api/...`

- `POST /api/...`

- Frontend adapters added:

- None / list adapters

- Backend gaps found:

- None / list gaps

UX States Covered

- [] Loading

- [] Empty

- [] Error

- [] Offline

- [] Permission denied

- [] Retry action

- [] Validation

- [] Success toast

Verification

- [] App builds

- [] Key routes render

- [] Forms validate

- [] API success path checked

- [] API failure path checked

- [] Responsive check completed

Notes for Tester

-

Final response format after implementation:

1. Implementation summary
2. Screens/routes implemented
3. API functions connected
4. Fallback states covered
5. Files changed
6. Verification performed with exact commands
7. Known backend gaps
8. Suggested commit message and PR title

H-- feat/matters-core-api

AI Prompt

Prompt from product plan, shortened:

You are a senior frontend engineer building BillSync Legal, a React + Tailwind legal-tech SaaS platform. Use the Stitch UI design source at E:\Shweta\UI Design\stitch_billsync_legal_platform_ui and the backend/API source at E:\Shweta\legal-billing-backend. The implementation must cover the complete BillSync frontend scope: roughly 75 product screens plus 24 fallback, loading, empty, and error states. The frontend must connect to real backend APIs where they exist, create clean frontend service adapters where backend APIs are missing, and document every backend gap clearly. The UI is for lawyers and law firm operators who are not technical, so never expose raw technical words to users such as API, backend, payload, token, 401, CORS, endpoint, database, stack trace, or server exception. Translate all failures into calm, plain-language guidance. Use the BillSync palette: background #F8F5EF, panels #FFFFFF, navigation #0B1F3A, primary #123B63, accent gold #C9A227, text #1F2933, secondary text #6B7280, border #E5E7EB, success #15803D, warning #D97706, error #B91C1C. Use subtle purposeful animation only: login spinner, live work-meter clock, pulsing recording dot, extension connected check, recorder waveform, upload progress, AI shimmer, payment spinner, and sync progress.

Branch to create/work on:

feat/matters-core-api

Screens/states/features to be added:

Matter list, create/edit, matter detail overview, matter team assignments

Design reference:

Matter overview and team screens

Backend/API reference:

Inspect E:\Shweta\legal-billing-backend for route inventory, models, validators, controllers, tests, and docs related to this branch.

How to design:

Use backend case and case-assignment APIs; keep user-facing terminology as Matter.

API connection direction:

Create or update frontend service functions for each real backend route used by this branch. Normalize responses into UI-safe models. If a backend route is missing, add a clearly named adapter and document the backend gap in the final response and PR.

Branch-specific senior-engineer direction:

Favor simple, composable implementation over clever abstractions. Keep UI states explicit and reusable. Connect real backend data first; use honest not-configured states for unavailable MVP expansion APIs.

Acceptance criteria:

Matter list/detail and assignments render; create/edit validates; empty states exist.

Implementation guardrails:

- Start by inspecting relevant Stitch screens in the design folder.
- Start by inspecting relevant backend route files, controllers, validators, models, tests, and docs.
- Create or update API client functions before wiring pages.
- Connect every available screen action to backend behavior where available.

- Where backend APIs are missing, add a service adapter with a clear TODO/backend-gap note and show honest not-connected/not-configured UI.
- Include loading, empty, error, offline, permission, retry, success, and validation states for every data surface.
- Keep lawyer-facing copy non-technical, calm, and action-oriented.
- Do not modify backend unless the branch explicitly asks for a tiny compatibility fix.
- Verify desktop, tablet, and 360px mobile width.

Verification required before marking this branch complete:

- Run npm install only if dependencies are missing.
- Run npm run dev and manually verify the branch route(s).
- Run npm run build before final response.
- Check at least one API success path and one API failure/fallback path.
- Check at least one empty state.
- Check no horizontal overflow at 360px.
- Confirm user-facing copy contains no technical backend jargon.

Commit message:

feat(matters): connect matter core screens

PR template before merging:

Summary

-

Screens / States Implemented

- []

API Mapping

- APIs connected:
 - `GET /api/...`
 - `POST /api/...`
- Frontend adapters added:
 - None / list adapters
- Backend gaps found:
 - None / list gaps

UX States Covered

- [] Loading
- [] Empty
- [] Error
- [] Offline
- [] Permission denied
- [] Retry action
- [] Validation
- [] Success toast

Verification

- [] App builds
- [] Key routes render
- [] Forms validate
- [] API success path checked
- [] API failure path checked
- [] Responsive check completed

Notes for Tester

-

Final response format after implementation:

1. Implementation summary
2. Screens/routes implemented
3. API functions connected
4. Fallback states covered
5. Files changed
6. Verification performed with exact commands
7. Known backend gaps
8. Suggested commit message and PR title

I-- feat/matter-timeline-docs-billing-audit

AI Prompt

Prompt from product plan, shortened:

You are a senior frontend engineer building BillSync Legal, a React + Tailwind legal-tech SaaS platform. Use the Stitch UI design source at E:\Shwetal\UI Design\stitch_billsync_legal_platform_ui and the backend/API source at E:\Shwetal\legal-billing-backend. The implementation must cover the complete BillSync frontend scope: roughly 75 product screens plus 24 fallback, loading, empty, and error states. The frontend must connect to real backend APIs where they exist, create clean frontend service adapters where backend APIs are missing, and document every backend gap clearly. The UI is for lawyers and law firm operators who are not technical, so never expose raw technical words to users such as API, backend, payload, token, 401, CORS, endpoint, database, stack trace, or server exception. Translate all failures into calm, plain-language guidance. Use the BillSync palette: background #F8F5EF, panels #FFFFFF, navigation #0B1F3A, primary #123B63, accent gold #C9A227, text #1F2933, secondary text #6B7280, border #E5E7EB, success #15803D, warning #D97706, error #B91C1C. Use subtle purposeful animation only: login spinner, live work-meter clock, pulsing recording dot, extension connected check, recorder waveform, upload progress, AI shimmer, payment spinner, and sync progress.

Branch to create/work on:

feat/matter-timeline-docs-billing-audit

Screens/states/features to be added:

Matter timeline, documents tab, billing tab, audit tab

Design reference:

Matter detail tabs

Backend/API reference:

Inspect E:\Shwetal\legal-billing-backend for route inventory, models, validators, controllers, tests, and docs related to this branch.

How to design:

Connect activities, time entries, billables, invoices, payments, documents/adapters, and audit/integration logs where available.

API connection direction:

Create or update frontend service functions for each real backend route used by this branch. Normalize responses into UI-safe models. If a backend route is missing, add a clearly named adapter and document the backend gap in the final response and PR.

Branch-specific senior-engineer direction:

Favor simple, composable implementation over clever abstractions. Keep UI states explicit and reusable. Connect real backend data first; use honest not-configured states for unavailable MVP expansion APIs.

Acceptance criteria:

Matter tabs render real/adapted data and honest missing-provider states.

Implementation guardrails:

- Start by inspecting relevant Stitch screens in the design folder.
- Start by inspecting relevant backend route files, controllers, validators, models, tests, and docs.
- Create or update API client functions before wiring pages.
- Connect every available screen action to backend behavior where available.
- Where backend APIs are missing, add a service adapter with a clear TODO/backend-gap note and show honest not-connected/not-configured UI.
- Include loading, empty, error, offline, permission, retry, success, and validation states for every data surface.
- Keep lawyer-facing copy non-technical, calm, and action-oriented.
- Do not modify backend unless the branch explicitly asks for a tiny compatibility fix.
- Verify desktop, tablet, and 360px mobile width.

Verification required before marking this branch complete:

- Run npm install only if dependencies are missing.
- Run npm run dev and manually verify the branch route(s).
- Run npm run build before final response.
- Check at least one API success path and one API failure/fallback path.
- Check at least one empty state.
- Check no horizontal overflow at 360px.
- Confirm user-facing copy contains no technical backend jargon.

Commit message:

feat(matters): add timeline documents billing audit tabs

PR template before merging:

Summary

-

Screens / States Implemented

- []

API Mapping

- APIs connected:

- `GET /api/...`

- `POST /api/...`

- Frontend adapters added:

- None / list adapters

- Backend gaps found:

- None / list gaps

UX States Covered

- [] Loading

- [] Empty

- [] Error

- [] Offline

- [] Permission denied

- [] Retry action

- [] Validation

- [] Success toast

Verification

- [] App builds

- [] Key routes render

- [] Forms validate

- [] API success path checked

- [] API failure path checked

- [] Responsive check completed

Notes for Tester

-

Final response format after implementation:

1. Implementation summary
2. Screens/routes implemented
3. API functions connected
4. Fallback states covered
5. Files changed
6. Verification performed with exact commands
7. Known backend gaps
8. Suggested commit message and PR title

J-- feat/tasks-daily-work

AI Prompt

Prompt from product plan, shortened:

You are a senior frontend engineer building BillSync Legal, a React + Tailwind legal-tech SaaS platform. Use the Stitch UI design source at E:\Shwetal\UI Design\stitch_billsync_legal_platform_ui and the backend/API source at E:\Shwetal\legal-billing-backend. The implementation must cover the complete BillSync frontend scope: roughly 75 product screens plus 24 fallback, loading, empty, and error states. The frontend must connect to real backend APIs where they exist, create clean frontend service adapters where backend APIs are missing, and document every backend gap clearly. The UI is for lawyers and law firm operators who are not technical, so never expose raw technical words to users such as API, backend, payload, token, 401, CORS, endpoint, database, stack trace, or server exception. Translate all failures into calm, plain-language guidance. Use the BillSync palette: background #F8F5EF, panels #FFFFFF, navigation #0B1F3A, primary #123B63, accent gold #C9A227, text #1F2933, secondary text #6B7280, border #E5E7EB, success #15803D, warning #D97706, error #B91C1C. Use subtle purposeful animation only: login spinner, live work-meter clock, pulsing recording dot, extension connected check, recorder waveform, upload progress, AI shimmer, payment spinner, and sync progress.

Branch to create/work on:

feat/tasks-daily-work

Screens/states/features to be added:

My Tasks, task board, task detail, task create/edit, My Work Today

Design reference:

Task and daily work screens

Backend/API reference:

Inspect E:\Shwetal\legal-billing-backend for route inventory, models, validators, controllers, tests, and docs related to this branch.

How to design:

If backend task APIs are missing, create adapters and document backend gaps. Connect task context into work meter.

API connection direction:

Create or update frontend service functions for each real backend route used by this branch. Normalize responses into UI-safe models. If a backend route is missing, add a clearly named adapter and document the backend gap in the final response and PR.

Branch-specific senior-engineer direction:

Favor simple, composable implementation over clever abstractions. Keep UI states explicit and reusable. Connect real backend data first; use honest not-configured states for unavailable MVP expansion APIs.

Acceptance criteria:

Task pages render; task detail can launch meter context; missing task backend is documented.

Implementation guardrails:

- Start by inspecting relevant Stitch screens in the design folder.
- Start by inspecting relevant backend route files, controllers, validators, models, tests, and docs.
- Create or update API client functions before wiring pages.

- Connect every available screen action to backend behavior where available.
- Where backend APIs are missing, add a service adapter with a clear TODO/backend-gap note and show honest not-connected/not-configured UI.
- Include loading, empty, error, offline, permission, retry, success, and validation states for every data surface.
- Keep lawyer-facing copy non-technical, calm, and action-oriented.
- Do not modify backend unless the branch explicitly asks for a tiny compatibility fix.
- Verify desktop, tablet, and 360px mobile width.

Verification required before marking this branch complete:

- Run npm install only if dependencies are missing.
- Run npm run dev and manually verify the branch route(s).
- Run npm run build before final response.
- Check at least one API success path and one API failure/fallback path.
- Check at least one empty state.
- Check no horizontal overflow at 360px.
- Confirm user-facing copy contains no technical backend jargon.

Commit message:

feat(tasks): implement daily task workflows

PR template before merging:

Summary

-

Screens / States Implemented

- []

API Mapping

- APIs connected:
 - `GET /api/...`
 - `POST /api/...`
- Frontend adapters added:
 - None / list adapters
- Backend gaps found:
 - None / list gaps

UX States Covered

- [] Loading
- [] Empty
- [] Error
- [] Offline
- [] Permission denied
- [] Retry action
- [] Validation
- [] Success toast

Verification

- [] App builds
- [] Key routes render
- [] Forms validate
- [] API success path checked
- [] API failure path checked
- [] Responsive check completed

Notes for Tester

-

Final response format after implementation:

1. Implementation summary
2. Screens/routes implemented
3. API functions connected
4. Fallback states covered
5. Files changed
6. Verification performed with exact commands
7. Known backend gaps
8. Suggested commit message and PR title

K-- feat/calendar-hearings

AI Prompt

Prompt from product plan, shortened:

You are a senior frontend engineer building BillSync Legal, a React + Tailwind legal-tech SaaS platform. Use the Stitch UI design source at E:\Shwetal\UI Design\stitch_billsync_legal_platform_ui and the backend/API source at E:\Shwetal\legal-billing-backend. The implementation must cover the complete BillSync frontend scope: roughly 75 product screens plus 24 fallback, loading, empty, and error states. The frontend must connect to real backend APIs where they exist, create clean frontend service adapters where backend APIs are missing, and document every backend gap clearly. The UI is for lawyers and law firm operators who are not technical, so never expose raw technical words to users such as API, backend, payload, token, 401, CORS, endpoint, database, stack trace, or server exception. Translate all failures into calm, plain-language guidance. Use the BillSync palette: background #F8F5EF, panels #FFFFFF, navigation #0B1F3A, primary #123B63, accent gold #C9A227, text #1F2933, secondary text #6B7280, border #E5E7EB, success #15803D, warning #D97706, error #B91C1C. Use subtle purposeful animation only: login spinner, live work-meter clock, pulsing recording dot, extension connected check, recorder waveform, upload progress, AI shimmer, payment spinner, and sync progress.

Branch to create/work on:

feat/calendar-hearings

Screens/states/features to be added:

Calendar, hearing dashboard, court hearing time entry, calendar not connected state

Design reference:

Calendar and hearing screens

Backend/API reference:

Inspect E:\Shwetal\legal-billing-backend for route inventory, models, validators, controllers, tests, and docs related to this branch.

How to design:

Wire available activity/time-entry APIs for manual hearing capture; use adapter for calendar provider if missing.

API connection direction:

Create or update frontend service functions for each real backend route used by this branch. Normalize responses into UI-safe models. If a backend route is missing, add a clearly named adapter and document the backend gap in the final response and PR.

Branch-specific senior-engineer direction:

Favor simple, composable implementation over clever abstractions. Keep UI states explicit and reusable. Connect real backend data first; use honest not-configured states for unavailable MVP expansion APIs.

Acceptance criteria:

Calendar/hearing pages render; hearing time saves where backend allows; calendar fallback exists.

Implementation guardrails:

- Start by inspecting relevant Stitch screens in the design folder.
- Start by inspecting relevant backend route files, controllers, validators, models, tests, and docs.
- Create or update API client functions before wiring pages.
- Connect every available screen action to backend behavior where available.
- Where backend APIs are missing, add a service adapter with a clear TODO/backend-gap note and show honest not-connected/not-configured UI.
- Include loading, empty, error, offline, permission, retry, success, and validation states for every data surface.
- Keep lawyer-facing copy non-technical, calm, and action-oriented.
- Do not modify backend unless the branch explicitly asks for a tiny compatibility fix.
- Verify desktop, tablet, and 360px mobile width.

Verification required before marking this branch complete:

- Run npm install only if dependencies are missing.
- Run npm run dev and manually verify the branch route(s).
- Run npm run build before final response.
- Check at least one API success path and one API failure/fallback path.
- Check at least one empty state.
- Check no horizontal overflow at 360px.
- Confirm user-facing copy contains no technical backend jargon.

Commit message:

feat(calendar): add hearing and calendar workflows

PR template before merging:

Summary

-

Screens / States Implemented

- []

API Mapping

- APIs connected:

- `GET /api/...`

- `POST /api/...`

- Frontend adapters added:

- None / list adapters

- Backend gaps found:

- None / list gaps

UX States Covered

- [] Loading

- [] Empty

- [] Error

- [] Offline

- [] Permission denied

- [] Retry action

- [] Validation

- [] Success toast

Verification

- [] App builds

- [] Key routes render

- [] Forms validate

- [] API success path checked

- [] API failure path checked

- [] Responsive check completed

Notes for Tester

-

Final response format after implementation:

1. Implementation summary
2. Screens/routes implemented
3. API functions connected
4. Fallback states covered
5. Files changed
6. Verification performed with exact commands
7. Known backend gaps
8. Suggested commit message and PR title

L-- feat/work-meter-time-capture

AI Prompt

Prompt from product plan, shortened:

You are a senior frontend engineer building BillSync Legal, a React + Tailwind legal-tech SaaS platform. Use the Stitch UI design source at E:\Shwetal\UI Design\stitch_billsync_legal_platform_ui and the backend/API source at E:\Shwetal\legal-billing-backend. The implementation must cover the complete BillSync frontend scope: roughly 75 product screens plus 24 fallback, loading, empty, and error states. The frontend must connect to real backend APIs where they exist, create clean frontend service adapters where backend APIs are missing, and document every backend gap clearly. The UI is for lawyers and law firm operators who are not technical, so never expose raw technical words to users such as API, backend, payload, token, 401, CORS, endpoint, database, stack trace, or server exception. Translate all failures into calm, plain-language guidance. Use the BillSync palette: background #F8F5EF, panels #FFFFFF, navigation #0B1F3A, primary #123B63, accent gold #C9A227, text #1F2933, secondary text #6B7280, border #E5E7EB, success #15803D, warning #D97706, error #B91C1C. Use subtle purposeful animation only: login spinner, live work-meter clock, pulsing recording dot, extension connected check, recorder waveform, upload progress, AI shimmer, payment spinner, and sync progress.

Branch to create/work on:

feat/work-meter-time-capture

Screens/states/features to be added:

Work Meter, work session history, manual time entry, captured work review, submit work for approval

Design reference:

Work meter and capture screens

Backend/API reference:

Inspect E:\Shwetal\legal-billing-backend for route inventory, models, validators, controllers, tests, and docs related to this branch.

How to design:

Wire work-session, time-entry, activity, and billable conversion pathways.

API connection direction:

Create or update frontend service functions for each real backend route used by this branch. Normalize responses into UI-safe models. If a backend route is missing, add a clearly named adapter and document the backend gap in the final response and PR.

Branch-specific senior-engineer direction:

Favor simple, composable implementation over clever abstractions. Keep UI states explicit and reusable. Connect real backend data first; use honest not-configured states for unavailable MVP expansion APIs.

Acceptance criteria:

Start/stop/save works; save-failed state preserves time; submit-for-approval flow exists.

Implementation guardrails:

- Start by inspecting relevant Stitch screens in the design folder.
- Start by inspecting relevant backend route files, controllers, validators, models, tests, and docs.
- Create or update API client functions before wiring pages.
- Connect every available screen action to backend behavior where available.

- Where backend APIs are missing, add a service adapter with a clear TODO/backend-gap note and show honest not-connected/not-configured UI.
- Include loading, empty, error, offline, permission, retry, success, and validation states for every data surface.
- Keep lawyer-facing copy non-technical, calm, and action-oriented.
- Do not modify backend unless the branch explicitly asks for a tiny compatibility fix.
- Verify desktop, tablet, and 360px mobile width.

Verification required before marking this branch complete:

- Run npm install only if dependencies are missing.
- Run npm run dev and manually verify the branch route(s).
- Run npm run build before final response.
- Check at least one API success path and one API failure/fallback path.
- Check at least one empty state.
- Check no horizontal overflow at 360px.
- Confirm user-facing copy contains no technical backend jargon.

Commit message:

feat(work): connect meter and time capture

PR template before merging:

Summary

-

Screens / States Implemented

- []

API Mapping

- APIs connected:
 - `GET /api/...`
 - `POST /api/...`
- Frontend adapters added:
 - None / list adapters
- Backend gaps found:
 - None / list gaps

UX States Covered

- [] Loading
- [] Empty
- [] Error
- [] Offline
- [] Permission denied
- [] Retry action
- [] Validation
- [] Success toast

Verification

- [] App builds
- [] Key routes render
- [] Forms validate
- [] API success path checked
- [] API failure path checked
- [] Responsive check completed

Notes for Tester

-

Final response format after implementation:

1. Implementation summary
2. Screens/routes implemented
3. API functions connected
4. Fallback states covered
5. Files changed
6. Verification performed with exact commands
7. Known backend gaps
8. Suggested commit message and PR title

M-- feat/chrome-extension-setup-status

AI Prompt

Prompt from product plan, shortened:

You are a senior frontend engineer building BillSync Legal, a React + Tailwind legal-tech SaaS platform. Use the Stitch UI design source at E:\Shwetal\UI Design\stitch_billsync_legal_platform_ui and the backend/API source at E:\Shwetal\legal-billing-backend. The implementation must cover the complete BillSync frontend scope: roughly 75 product screens plus 24 fallback, loading, empty, and error states. The frontend must connect to real backend APIs where they exist, create clean frontend service adapters where backend APIs are missing, and document every backend gap clearly. The UI is for lawyers and law firm operators who are not technical, so never expose raw technical words to users such as API, backend, payload, token, 401, CORS, endpoint, database, stack trace, or server exception. Translate all failures into calm, plain-language guidance. Use the BillSync palette: background #F8F5EF, panels #FFFFFF, navigation #0B1F3A, primary #123B63, accent gold #C9A227, text #1F2933, secondary text #6B7280, border #E5E7EB, success #15803D, warning #D97706, error #B91C1C. Use subtle purposeful animation only: login spinner, live work-meter clock, pulsing recording dot, extension connected check, recorder waveform, upload progress, AI shimmer, payment spinner, and sync progress.

Branch to create/work on:

feat/chrome-extension-setup-status

Screens/states/features to be added:

Chrome extension setup, extension status, extension troubleshooting

Design reference:

Extension setup/status screens

Backend/API reference:

Inspect E:\Shwetal\legal-billing-backend for route inventory, models, validators, controllers, tests, and docs related to this branch.

How to design:

Use extension README/QA docs and email-entry backend behavior. Make load-unpacked setup lawyer-friendly.

API connection direction:

Create or update frontend service functions for each real backend route used by this branch. Normalize responses into UI-safe models. If a backend route is missing, add a clearly named adapter and document the backend gap in the final response and PR.

Branch-specific senior-engineer direction:

Favor simple, composable implementation over clever abstractions. Keep UI states explicit and reusable. Connect real backend data first; use honest not-configured states for unavailable MVP expansion APIs.

Acceptance criteria:

Setup guide and status cards render; test actions are available; extension not connected states exist.

Implementation guardrails:

- Start by inspecting relevant Stitch screens in the design folder.
- Start by inspecting relevant backend route files, controllers, validators, models, tests, and docs.
- Create or update API client functions before wiring pages.
- Connect every available screen action to backend behavior where available.
- Where backend APIs are missing, add a service adapter with a clear TODO/backend-gap note and show honest not-connected/not-configured UI.
- Include loading, empty, error, offline, permission, retry, success, and validation states for every data surface.
- Keep lawyer-facing copy non-technical, calm, and action-oriented.
- Do not modify backend unless the branch explicitly asks for a tiny compatibility fix.
- Verify desktop, tablet, and 360px mobile width.

Verification required before marking this branch complete:

- Run npm install only if dependencies are missing.
- Run npm run dev and manually verify the branch route(s).
- Run npm run build before final response.
- Check at least one API success path and one API failure/fallback path.
- Check at least one empty state.
- Check no horizontal overflow at 360px.
- Confirm user-facing copy contains no technical backend jargon.

Commit message:

feat(extension): add setup status troubleshooting

PR template before merging:

Summary

-

Screens / States Implemented

- []

API Mapping

- APIs connected:

- `GET /api/...`

- `POST /api/...`

- Frontend adapters added:

- None / list adapters

- Backend gaps found:

- None / list gaps

UX States Covered

- [] Loading

- [] Empty

- [] Error

- [] Offline

- [] Permission denied

- [] Retry action

- [] Validation

- [] Success toast

Verification

- [] App builds

- [] Key routes render

- [] Forms validate

- [] API success path checked

- [] API failure path checked

- [] Responsive check completed

Notes for Tester

-

Final response format after implementation:

1. Implementation summary
2. Screens/routes implemented
3. API functions connected
4. Fallback states covered
5. Files changed
6. Verification performed with exact commands
7. Known backend gaps
8. Suggested commit message and PR title

N-- feat/email-research-capture

AI Prompt

Prompt from product plan, shortened:

You are a senior frontend engineer building BillSync Legal, a React + Tailwind legal-tech SaaS platform. Use the Stitch UI design source at E:\Shwetal\UI Design\stitch_billsync_legal_platform_ui and the backend/API source at E:\Shwetal\legal-billing-backend. The implementation must cover the complete BillSync frontend scope: roughly 75 product screens plus 24 fallback, loading, empty, and error states. The frontend must connect to real backend APIs where they exist, create clean frontend service adapters where backend APIs are missing, and document every backend gap clearly. The UI is for lawyers and law firm operators who are not technical, so never expose raw technical words to users such as API, backend, payload, token, 401, CORS, endpoint, database, stack trace, or server exception. Translate all failures into calm, plain-language guidance. Use the BillSync palette: background #F8F5EF, panels #FFFFFF, navigation #0B1F3A, primary #123B63, accent gold #C9A227, text #1F2933, secondary text #6B7280, border #E5E7EB, success #15803D, warning #D97706, error #B91C1C. Use subtle purposeful animation only: login spinner, live work-meter clock, pulsing recording dot, extension connected check, recorder waveform, upload progress, AI shimmer, payment spinner, and sync progress.

Branch to create/work on:

feat/email-research-capture

Screens/states/features to be added:

Gmail capture review, research capture review, capture mapping, conversion to billable/time where available

Design reference:

Captured email/research screens

Backend/API reference:

Inspect E:\Shwetal\legal-billing-backend for route inventory, models, validators, controllers, tests, and docs related to this branch.

How to design:

Wire email-entry APIs and conversion/map behavior.

API connection direction:

Create or update frontend service functions for each real backend route used by this branch. Normalize responses into UI-safe models. If a backend route is missing, add a clearly named adapter and document the backend gap in the final response and PR.

Branch-specific senior-engineer direction:

Favor simple, composable implementation over clever abstractions. Keep UI states explicit and reusable. Connect real backend data first; use honest not-configured states for unavailable MVP expansion APIs.

Acceptance criteria:

Captured lists render; map to client/matter works; converted/failed/empty states exist.

Implementation guardrails:

- Start by inspecting relevant Stitch screens in the design folder.
- Start by inspecting relevant backend route files, controllers, validators, models, tests, and docs.
- Create or update API client functions before wiring pages.
- Connect every available screen action to backend behavior where available.

- Where backend APIs are missing, add a service adapter with a clear TODO/backend-gap note and show honest not-connected/not-configured UI.
- Include loading, empty, error, offline, permission, retry, success, and validation states for every data surface.
- Keep lawyer-facing copy non-technical, calm, and action-oriented.
- Do not modify backend unless the branch explicitly asks for a tiny compatibility fix.
- Verify desktop, tablet, and 360px mobile width.

Verification required before marking this branch complete:

- Run npm install only if dependencies are missing.
- Run npm run dev and manually verify the branch route(s).
- Run npm run build before final response.
- Check at least one API success path and one API failure/fallback path.
- Check at least one empty state.
- Check no horizontal overflow at 360px.
- Confirm user-facing copy contains no technical backend jargon.

Commit message:

feat(capture): connect Gmail and research review

PR template before merging:

Summary

-

Screens / States Implemented

- []

API Mapping

- APIs connected:
 - `GET /api/...`
 - `POST /api/...`
- Frontend adapters added:
 - None / list adapters
- Backend gaps found:
 - None / list gaps

UX States Covered

- [] Loading
- [] Empty
- [] Error
- [] Offline
- [] Permission denied
- [] Retry action
- [] Validation
- [] Success toast

Verification

- [] App builds
- [] Key routes render
- [] Forms validate
- [] API success path checked
- [] API failure path checked
- [] Responsive check completed

Notes for Tester

-

Final response format after implementation:

1. Implementation summary
2. Screens/routes implemented
3. API functions connected
4. Fallback states covered
5. Files changed
6. Verification performed with exact commands
7. Known backend gaps
8. Suggested commit message and PR title

O-- feat/call-meeting-recorder

AI Prompt

Prompt from product plan, shortened:

You are a senior frontend engineer building BillSync Legal, a React + Tailwind legal-tech SaaS platform. Use the Stitch UI design source at E:\Shwetal\UI Design\stitch_billsync_legal_platform_ui and the backend/API source at E:\Shwetal\legal-billing-backend. The implementation must cover the complete BillSync frontend scope: roughly 75 product screens plus 24 fallback, loading, empty, and error states. The frontend must connect to real backend APIs where they exist, create clean frontend service adapters where backend APIs are missing, and document every backend gap clearly. The UI is for lawyers and law firm operators who are not technical, so never expose raw technical words to users such as API, backend, payload, token, 401, CORS, endpoint, database, stack trace, or server exception. Translate all failures into calm, plain-language guidance. Use the BillSync palette: background #F8F5EF, panels #FFFFFF, navigation #0B1F3A, primary #123B63, accent gold #C9A227, text #1F2933, secondary text #6B7280, border #E5E7EB, success #15803D, warning #D97706, error #B91C1C. Use subtle purposeful animation only: login spinner, live work-meter clock, pulsing recording dot, extension connected check, recorder waveform, upload progress, AI shimmer, payment spinner, and sync progress.

Branch to create/work on:

feat/call-meeting-recorder

Screens/states/features to be added:

Call/meeting recorder, recording library, recording detail with transcript and matter link

Design reference:

Recorder screens

Backend/API reference:

Inspect E:\Shwetal\legal-billing-backend for route inventory, models, validators, controllers, tests, and docs related to this branch.

How to design:

If recorder APIs are missing, build honest not-configured UI and future-ready adapters.

API connection direction:

Create or update frontend service functions for each real backend route used by this branch. Normalize responses into UI-safe models. If a backend route is missing, add a clearly named adapter and document the backend gap in the final response and PR.

Branch-specific senior-engineer direction:

Favor simple, composable implementation over clever abstractions. Keep UI states explicit and reusable. Connect real backend data first; use honest not-configured states for unavailable MVP expansion APIs.

Acceptance criteria:

Recorder pages render; permission blocked/unavailable states exist; no fake recording persistence.

Implementation guardrails:

- Start by inspecting relevant Stitch screens in the design folder.
- Start by inspecting relevant backend route files, controllers, validators, models, tests, and docs.
- Create or update API client functions before wiring pages.
- Connect every available screen action to backend behavior where available.
- Where backend APIs are missing, add a service adapter with a clear TODO/backend-gap note and show honest not-connected/not-configured UI.
- Include loading, empty, error, offline, permission, retry, success, and validation states for every data surface.
- Keep lawyer-facing copy non-technical, calm, and action-oriented.
- Do not modify backend unless the branch explicitly asks for a tiny compatibility fix.
- Verify desktop, tablet, and 360px mobile width.

Verification required before marking this branch complete:

- Run npm install only if dependencies are missing.
- Run npm run dev and manually verify the branch route(s).
- Run npm run build before final response.
- Check at least one API success path and one API failure/fallback path.
- Check at least one empty state.
- Check no horizontal overflow at 360px.
- Confirm user-facing copy contains no technical backend jargon.

Commit message:

feat(recorder): add call meeting recorder screens

PR template before merging:

Summary

-

Screens / States Implemented

- []

API Mapping

- APIs connected:

- `GET /api/...`

- `POST /api/...`

- Frontend adapters added:

- None / list adapters

- Backend gaps found:

- None / list gaps

UX States Covered

- [] Loading

- [] Empty

- [] Error

- [] Offline

- [] Permission denied

- [] Retry action

- [] Validation

- [] Success toast

Verification

- [] App builds

- [] Key routes render

- [] Forms validate

- [] API success path checked

- [] API failure path checked

- [] Responsive check completed

Notes for Tester

-

Final response format after implementation:

1. Implementation summary
2. Screens/routes implemented
3. API functions connected
4. Fallback states covered
5. Files changed
6. Verification performed with exact commands
7. Known backend gaps
8. Suggested commit message and PR title

P-- feat/billables-rates-api

AI Prompt

Prompt from product plan, shortened:

You are a senior frontend engineer building BillSync Legal, a React + Tailwind legal-tech SaaS platform. Use the Stitch UI design source at E:\Shweta\UI Design\stitch_billsync_legal_platform_ui and the backend/API source at E:\Shweta\legal-billing-backend. The implementation must cover the complete BillSync frontend scope: roughly 75 product screens plus 24 fallback, loading, empty, and error states. The frontend must connect to real backend APIs where they exist, create clean frontend service adapters where backend APIs are missing, and document every backend gap clearly. The UI is for lawyers and law firm operators who are not technical, so never expose raw technical words to users such as API, backend, payload, token, 401, CORS, endpoint, database, stack trace, or server exception. Translate all failures into calm, plain-language guidance. Use the BillSync palette: background #F8F5EF, panels #FFFFFF, navigation #0B1F3A, primary #123B63, accent gold #C9A227, text #1F2933, secondary text #6B7280, border #E5E7EB, success #15803D, warning #D97706, error #B91C1C. Use subtle purposeful animation only: login spinner, live work-meter clock, pulsing recording dot, extension connected check, recorder waveform, upload progress, AI shimmer, payment spinner, and sync progress.

Branch to create/work on:

feat/billables-rates-api

Screens/states/features to be added:

Billables list, billable detail, approval queue, rate cards

Design reference:

Billable/rate screens

Backend/API reference:

Inspect E:\Shweta\legal-billing-backend for route inventory, models, validators, controllers, tests, and docs related to this branch.

How to design:

Wire billables and rate-card APIs.

API connection direction:

Create or update frontend service functions for each real backend route used by this branch. Normalize responses into UI-safe models. If a backend route is missing, add a clearly named adapter and document the backend gap in the final response and PR.

Branch-specific senior-engineer direction:

Favor simple, composable implementation over clever abstractions. Keep UI states explicit and reusable. Connect real backend data first; use honest not-configured states for unavailable MVP expansion APIs.

Acceptance criteria:

Billable filters, approval queue, detail, and rate cards work with success/failure states.

Implementation guardrails:

- Start by inspecting relevant Stitch screens in the design folder.
- Start by inspecting relevant backend route files, controllers, validators, models, tests, and docs.
- Create or update API client functions before wiring pages.
- Connect every available screen action to backend behavior where available.

- Where backend APIs are missing, add a service adapter with a clear TODO/backend-gap note and show honest not-connected/not-configured UI.
- Include loading, empty, error, offline, permission, retry, success, and validation states for every data surface.
- Keep lawyer-facing copy non-technical, calm, and action-oriented.
- Do not modify backend unless the branch explicitly asks for a tiny compatibility fix.
- Verify desktop, tablet, and 360px mobile width.

Verification required before marking this branch complete:

- Run npm install only if dependencies are missing.
- Run npm run dev and manually verify the branch route(s).
- Run npm run build before final response.
- Check at least one API success path and one API failure/fallback path.
- Check at least one empty state.
- Check no horizontal overflow at 360px.
- Confirm user-facing copy contains no technical backend jargon.

Commit message:

feat(billing): connect billables and rates

PR template before merging:

Summary

-

Screens / States Implemented

- []

API Mapping

- APIs connected:
 - `GET /api/...`
 - `POST /api/...`
- Frontend adapters added:
 - None / list adapters
- Backend gaps found:
 - None / list gaps

UX States Covered

- [] Loading
- [] Empty
- [] Error
- [] Offline
- [] Permission denied
- [] Retry action
- [] Validation
- [] Success toast

Verification

- [] App builds
- [] Key routes render
- [] Forms validate
- [] API success path checked
- [] API failure path checked
- [] Responsive check completed

Notes for Tester

-

Final response format after implementation:

1. Implementation summary
2. Screens/routes implemented
3. API functions connected
4. Fallback states covered
5. Files changed
6. Verification performed with exact commands
7. Known backend gaps
8. Suggested commit message and PR title

Q-- feat/invoices-api

AI Prompt

Prompt from product plan, shortened:

You are a senior frontend engineer building BillSync Legal, a React + Tailwind legal-tech SaaS platform. Use the Stitch UI design source at E:\Shwetal\UI Design\stitch_billsync_legal_platform_ui and the backend/API source at E:\Shwetal\legal-billing-backend. The implementation must cover the complete BillSync frontend scope: roughly 75 product screens plus 24 fallback, loading, empty, and error states. The frontend must connect to real backend APIs where they exist, create clean frontend service adapters where backend APIs are missing, and document every backend gap clearly. The UI is for lawyers and law firm operators who are not technical, so never expose raw technical words to users such as API, backend, payload, token, 401, CORS, endpoint, database, stack trace, or server exception. Translate all failures into calm, plain-language guidance. Use the BillSync palette: background #F8F5EF, panels #FFFFFF, navigation #0B1F3A, primary #123B63, accent gold #C9A227, text #1F2933, secondary text #6B7280, border #E5E7EB, success #15803D, warning #D97706, error #B91C1C. Use subtle purposeful animation only: login spinner, live work-meter clock, pulsing recording dot, extension connected check, recorder waveform, upload progress, AI shimmer, payment spinner, and sync progress.

Branch to create/work on:

feat/invoices-api

Screens/states/features to be added:

Invoice list, invoice builder, invoice detail, invoice send/share, invoice templates

Design reference:

Invoice screens

Backend/API reference:

Inspect E:\Shwetal\legal-billing-backend for route inventory, models, validators, controllers, tests, and docs related to this branch.

How to design:

Wire invoice APIs, from-billables/from-time generation, status actions, and template shell.

API connection direction:

Create or update frontend service functions for each real backend route used by this branch. Normalize responses into UI-safe models. If a backend route is missing, add a clearly named adapter and document the backend gap in the final response and PR.

Branch-specific senior-engineer direction:

Favor simple, composable implementation over clever abstractions. Keep UI states explicit and reusable. Connect real backend data first; use honest not-configured states for unavailable MVP expansion APIs.

Acceptance criteria:

Invoice list/detail/builder render; generation failure state exists; no fake real email/PDF claim.

Implementation guardrails:

- Start by inspecting relevant Stitch screens in the design folder.
- Start by inspecting relevant backend route files, controllers, validators, models, tests, and docs.
- Create or update API client functions before wiring pages.
- Connect every available screen action to backend behavior where available.
- Where backend APIs are missing, add a service adapter with a clear TODO/backend-gap note and show honest not-connected/not-configured UI.
- Include loading, empty, error, offline, permission, retry, success, and validation states for every data surface.
- Keep lawyer-facing copy non-technical, calm, and action-oriented.
- Do not modify backend unless the branch explicitly asks for a tiny compatibility fix.
- Verify desktop, tablet, and 360px mobile width.

Verification required before marking this branch complete:

- Run npm install only if dependencies are missing.
- Run npm run dev and manually verify the branch route(s).
- Run npm run build before final response.
- Check at least one API success path and one API failure/fallback path.
- Check at least one empty state.
- Check no horizontal overflow at 360px.
- Confirm user-facing copy contains no technical backend jargon.

Commit message:

feat(invoices): connect invoice workflows

PR template before merging:

Summary

-

Screens / States Implemented

- []

API Mapping

- APIs connected:

- `GET /api/...`

- `POST /api/...`

- Frontend adapters added:

- None / list adapters

- Backend gaps found:

- None / list gaps

UX States Covered

- [] Loading

- [] Empty

- [] Error

- [] Offline

- [] Permission denied

- [] Retry action

- [] Validation

- [] Success toast

Verification

- [] App builds

- [] Key routes render

- [] Forms validate

- [] API success path checked

- [] API failure path checked

- [] Responsive check completed

Notes for Tester

-

Final response format after implementation:

1. Implementation summary
2. Screens/routes implemented
3. API functions connected
4. Fallback states covered
5. Files changed
6. Verification performed with exact commands
7. Known backend gaps
8. Suggested commit message and PR title

R-- feat/payments-portal-reconciliation

AI Prompt

Prompt from product plan, shortened:

You are a senior frontend engineer building BillSync Legal, a React + Tailwind legal-tech SaaS platform. Use the Stitch UI design source at E:\Shweta\UI Design\stitch_billsync_legal_platform_ui and the backend/API source at E:\Shweta\legal-billing-backend. The implementation must cover the complete BillSync frontend scope: roughly 75 product screens plus 24 fallback, loading, empty, and error states. The frontend must connect to real backend APIs where they exist, create clean frontend service adapters where backend APIs are missing, and document every backend gap clearly. The UI is for lawyers and law firm operators who are not technical, so never expose raw technical words to users such as API, backend, payload, token, 401, CORS, endpoint, database, stack trace, or server exception. Translate all failures into calm, plain-language guidance. Use the BillSync palette: background #F8F5EF, panels #FFFFFF, navigation #0B1F3A, primary #123B63, accent gold #C9A227, text #1F2933, secondary text #6B7280, border #E5E7EB, success #15803D, warning #D97706, error #B91C1C. Use subtle purposeful animation only: login spinner, live work-meter clock, pulsing recording dot, extension connected check, recorder waveform, upload progress, AI shimmer, payment spinner, and sync progress.

Branch to create/work on:

feat/payments-portal-reconciliation

Screens/states/features to be added:

Payment dashboard, payment entry, client payment portal, reconciliation, payment failed state

Design reference:

Payment screens

Backend/API reference:

Inspect E:\Shweta\legal-billing-backend for route inventory, models, validators, controllers, tests, and docs related to this branch.

How to design:

Wire payment APIs. If external gateway is missing, show portal setup/not-connected state.

API connection direction:

Create or update frontend service functions for each real backend route used by this branch. Normalize responses into UI-safe models. If a backend route is missing, add a clearly named adapter and document the backend gap in the final response and PR.

Branch-specific senior-engineer direction:

Favor simple, composable implementation over clever abstractions. Keep UI states explicit and reusable. Connect real backend data first; use honest not-configured states for unavailable MVP expansion APIs.

Acceptance criteria:

Payment record/reconcile works where backend allows; portal state is honest; failure states exist.

Implementation guardrails:

- Start by inspecting relevant Stitch screens in the design folder.
- Start by inspecting relevant backend route files, controllers, validators, models, tests, and docs.
- Create or update API client functions before wiring pages.
- Connect every available screen action to backend behavior where available.

- Where backend APIs are missing, add a service adapter with a clear TODO/backend-gap note and show honest not-connected/not-configured UI.
- Include loading, empty, error, offline, permission, retry, success, and validation states for every data surface.
- Keep lawyer-facing copy non-technical, calm, and action-oriented.
- Do not modify backend unless the branch explicitly asks for a tiny compatibility fix.
- Verify desktop, tablet, and 360px mobile width.

Verification required before marking this branch complete:

- Run npm install only if dependencies are missing.
- Run npm run dev and manually verify the branch route(s).
- Run npm run build before final response.
- Check at least one API success path and one API failure/fallback path.
- Check at least one empty state.
- Check no horizontal overflow at 360px.
- Confirm user-facing copy contains no technical backend jargon.

Commit message:

feat(payments): add payment portal and reconciliation

PR template before merging:

Summary

-

Screens / States Implemented

- []

API Mapping

- APIs connected:
 - `GET /api/...`
 - `POST /api/...`
- Frontend adapters added:
 - None / list adapters
- Backend gaps found:
 - None / list gaps

UX States Covered

- [] Loading
- [] Empty
- [] Error
- [] Offline
- [] Permission denied
- [] Retry action
- [] Validation
- [] Success toast

Verification

- [] App builds
- [] Key routes render
- [] Forms validate
- [] API success path checked
- [] API failure path checked
- [] Responsive check completed

Notes for Tester

-

Final response format after implementation:

1. Implementation summary
2. Screens/routes implemented
3. API functions connected
4. Fallback states covered
5. Files changed
6. Verification performed with exact commands
7. Known backend gaps
8. Suggested commit message and PR title

S-- feat/finance-reports-audit

AI Prompt

Prompt from product plan, shortened:

You are a senior frontend engineer building BillSync Legal, a React + Tailwind legal-tech SaaS platform. Use the Stitch UI design source at E:\Shweta\UI Design\stitch_billsync_legal_platform_ui and the backend/API source at E:\Shweta\legal-billing-backend. The implementation must cover the complete BillSync frontend scope: roughly 75 product screens plus 24 fallback, loading, empty, and error states. The frontend must connect to real backend APIs where they exist, create clean frontend service adapters where backend APIs are missing, and document every backend gap clearly. The UI is for lawyers and law firm operators who are not technical, so never expose raw technical words to users such as API, backend, payload, token, 401, CORS, endpoint, database, stack trace, or server exception. Translate all failures into calm, plain-language guidance. Use the BillSync palette: background #F8F5EF, panels #FFFFFF, navigation #0B1F3A, primary #123B63, accent gold #C9A227, text #1F2933, secondary text #6B7280, border #E5E7EB, success #15803D, warning #D97706, error #B91C1C. Use subtle purposeful animation only: login spinner, live work-meter clock, pulsing recording dot, extension connected check, recorder waveform, upload progress, AI shimmer, payment spinner, and sync progress.

Branch to create/work on:

feat/finance-reports-audit

Screens/states/features to be added:

Finance dashboard, reports, audit logs, AR/WIP/revenue/utilization views

Design reference:

Finance, reports, audit screens

Backend/API reference:

Inspect E:\Shwetal\legal-billing-backend for route inventory, models, validators, controllers, tests, and docs related to this branch.

How to design:

Wire analytics, revenue, KPI, reports, payments, invoices, and integration logs.

API connection direction:

Create or update frontend service functions for each real backend route used by this branch. Normalize responses into UI-safe models. If a backend route is missing, add a clearly named adapter and document the backend gap in the final response and PR.

Branch-specific senior-engineer direction:

Favor simple, composable implementation over clever abstractions. Keep UI states explicit and reusable. Connect real backend data first; use honest not-configured states for unavailable MVP expansion APIs.

Acceptance criteria:

Finance KPIs render; report exports trigger where available; audit logs show available events.

Implementation guardrails:

- Start by inspecting relevant Stitch screens in the design folder.
- Start by inspecting relevant backend route files, controllers, validators, models, tests, and docs.
- Create or update API client functions before wiring pages.
- Connect every available screen action to backend behavior where available.
- Where backend APIs are missing, add a service adapter with a clear TODO/backend-gap note and show honest not-connected/not-configured UI.
- Include loading, empty, error, offline, permission, retry, success, and validation states for every data surface.
- Keep lawyer-facing copy non-technical, calm, and action-oriented.
- Do not modify backend unless the branch explicitly asks for a tiny compatibility fix.
- Verify desktop, tablet, and 360px mobile width.

Verification required before marking this branch complete:

- Run npm install only if dependencies are missing.
- Run npm run dev and manually verify the branch route(s).
- Run npm run build before final response.
- Check at least one API success path and one API failure/fallback path.
- Check at least one empty state.
- Check no horizontal overflow at 360px.
- Confirm user-facing copy contains no technical backend jargon.

Commit message:

feat(finance): connect reports and audit screens

PR template before merging:

Summary

-

Screens / States Implemented

- []

API Mapping

- APIs connected:

- `GET /api/...`

- `POST /api/...`

- Frontend adapters added:

- None / list adapters

- Backend gaps found:

- None / list gaps

UX States Covered

- [] Loading

- [] Empty

- [] Error

- [] Offline

- [] Permission denied

- [] Retry action

- [] Validation

- [] Success toast

Verification

- [] App builds

- [] Key routes render

- [] Forms validate

- [] API success path checked

- [] API failure path checked

- [] Responsive check completed

Notes for Tester

-

Final response format after implementation:

1. Implementation summary
2. Screens/routes implemented
3. API functions connected
4. Fallback states covered
5. Files changed
6. Verification performed with exact commands
7. Known backend gaps
8. Suggested commit message and PR title

T-- feat/gst-tds-tax

AI Prompt

Prompt from product plan, shortened:

You are a senior frontend engineer building BillSync Legal, a React + Tailwind legal-tech SaaS platform. Use the Stitch UI design source at E:\Shwetal\UI Design\stitch_billsync_legal_platform_ui and the backend/API source at E:\Shwetal\legal-billing-backend. The implementation must cover the complete BillSync frontend scope: roughly 75 product screens plus 24 fallback, loading, empty, and error states. The frontend must connect to real backend APIs where they exist, create clean frontend service adapters where backend APIs are missing, and document every backend gap clearly. The UI is for lawyers and law firm operators who are not technical, so never expose raw technical words to users such as API, backend, payload, token, 401, CORS, endpoint, database, stack trace, or server exception. Translate all failures into calm, plain-language guidance. Use the BillSync palette: background #F8F5EF, panels #FFFFFF, navigation #0B1F3A, primary #123B63, accent gold #C9A227, text #1F2933, secondary text #6B7280, border #E5E7EB, success #15803D, warning #D97706, error #B91C1C. Use subtle purposeful animation only: login spinner, live work-meter clock, pulsing recording dot, extension connected check, recorder waveform, upload progress, AI shimmer, payment spinner, and sync progress.

Branch to create/work on:

feat/gst-tds-tax

Screens/states/features to be added:

GST dashboard/settings, TDS dashboard/settings, tax summaries and not-configured states

Design reference:

GST/TDS screens

Backend/API reference:

Inspect E:\Shwetal\legal-billing-backend for route inventory, models, validators, controllers, tests, and docs related to this branch.

How to design:

Wire firm tax settings and invoice tax fields where available. Use adapter for TDS if backend missing.

API connection direction:

Create or update frontend service functions for each real backend route used by this branch. Normalize responses into UI-safe models. If a backend route is missing, add a clearly named adapter and document the backend gap in the final response and PR.

Branch-specific senior-engineer direction:

Favor simple, composable implementation over clever abstractions. Keep UI states explicit and reusable. Connect real backend data first; use honest not-configured states for unavailable MVP expansion APIs.

Acceptance criteria:

GST settings render; TDS not-configured state exists if no API; tax pages avoid fake compliance.

Implementation guardrails:

- Start by inspecting relevant Stitch screens in the design folder.
- Start by inspecting relevant backend route files, controllers, validators, models, tests, and docs.
- Create or update API client functions before wiring pages.
- Connect every available screen action to backend behavior where available.

- Where backend APIs are missing, add a service adapter with a clear TODO/backend-gap note and show honest not-connected/not-configured UI.
- Include loading, empty, error, offline, permission, retry, success, and validation states for every data surface.
- Keep lawyer-facing copy non-technical, calm, and action-oriented.
- Do not modify backend unless the branch explicitly asks for a tiny compatibility fix.
- Verify desktop, tablet, and 360px mobile width.

Verification required before marking this branch complete:

- Run npm install only if dependencies are missing.
- Run npm run dev and manually verify the branch route(s).
- Run npm run build before final response.
- Check at least one API success path and one API failure/fallback path.
- Check at least one empty state.
- Check no horizontal overflow at 360px.
- Confirm user-facing copy contains no technical backend jargon.

Commit message:

feat(tax): add GST and TDS screens

PR template before merging:

Summary

-

Screens / States Implemented

- []

API Mapping

- APIs connected:
 - `GET /api/...`
 - `POST /api/...`
- Frontend adapters added:
 - None / list adapters
- Backend gaps found:
 - None / list gaps

UX States Covered

- [] Loading
- [] Empty
- [] Error
- [] Offline
- [] Permission denied
- [] Retry action
- [] Validation
- [] Success toast

Verification

- [] App builds
- [] Key routes render
- [] Forms validate
- [] API success path checked
- [] API failure path checked
- [] Responsive check completed

Notes for Tester

-

Final response format after implementation:

1. Implementation summary
2. Screens/routes implemented
3. API functions connected
4. Fallback states covered
5. Files changed
6. Verification performed with exact commands
7. Known backend gaps
8. Suggested commit message and PR title

U-- feat/hr-people

AI Prompt

Prompt from product plan, shortened:

You are a senior frontend engineer building BillSync Legal, a React + Tailwind legal-tech SaaS platform. Use the Stitch UI design source at E:\Shwetal\UI Design\stitch_billsync_legal_platform_ui and the backend/API source at E:\Shwetal\legal-billing-backend. The implementation must cover the complete BillSync frontend scope: roughly 75 product screens plus 24 fallback, loading, empty, and error states. The frontend must connect to real backend APIs where they exist, create clean frontend service adapters where backend APIs are missing, and document every backend gap clearly. The UI is for lawyers and law firm operators who are not technical, so never expose raw technical words to users such as API, backend, payload, token, 401, CORS, endpoint, database, stack trace, or server exception. Translate all failures into calm, plain-language guidance. Use the BillSync palette: background #F8F5EF, panels #FFFFFF, navigation #0B1F3A, primary #123B63, accent gold #C9A227, text #1F2933, secondary text #6B7280, border #E5E7EB, success #15803D, warning #D97706, error #B91C1C. Use subtle purposeful animation only: login spinner, live work-meter clock, pulsing recording dot, extension connected check, recorder waveform, upload progress, AI shimmer, payment spinner, and sync progress.

Branch to create/work on:

feat/hr-people

Screens/states/features to be added:

HR dashboard, team directory, employee profile, workload/attendance overview

Design reference:

HR/people screens

Backend/API reference:

Inspect E:\Shwetal\legal-billing-backend for route inventory, models, validators, controllers, tests, and docs related to this branch.

How to design:

Wire users/profiles where available and use workload from time/work/session data.

API connection direction:

Create or update frontend service functions for each real backend route used by this branch. Normalize responses into UI-safe models. If a backend route is missing, add a clearly named adapter and document the backend gap in the final response and PR.

Branch-specific senior-engineer direction:

Favor simple, composable implementation over clever abstractions. Keep UI states explicit and reusable. Connect real backend data first; use honest not-configured states for unavailable MVP expansion APIs.

Acceptance criteria:

People screens render real users; workload/adapters are clear; empty states exist.

Implementation guardrails:

- Start by inspecting relevant Stitch screens in the design folder.
- Start by inspecting relevant backend route files, controllers, validators, models, tests, and docs.
- Create or update API client functions before wiring pages.
- Connect every available screen action to backend behavior where available.
- Where backend APIs are missing, add a service adapter with a clear TODO/backend-gap note and show honest not-connected/not-configured UI.
- Include loading, empty, error, offline, permission, retry, success, and validation states for every data surface.
- Keep lawyer-facing copy non-technical, calm, and action-oriented.
- Do not modify backend unless the branch explicitly asks for a tiny compatibility fix.
- Verify desktop, tablet, and 360px mobile width.

Verification required before marking this branch complete:

- Run npm install only if dependencies are missing.
- Run npm run dev and manually verify the branch route(s).
- Run npm run build before final response.
- Check at least one API success path and one API failure/fallback path.
- Check at least one empty state.
- Check no horizontal overflow at 360px.
- Confirm user-facing copy contains no technical backend jargon.

Commit message:

feat(hr): add people and workload screens

PR template before merging:

Summary

-

Screens / States Implemented

- []

API Mapping

- APIs connected:

- `GET /api/...`

- `POST /api/...`

- Frontend adapters added:

- None / list adapters

- Backend gaps found:

- None / list gaps

UX States Covered

- [] Loading

- [] Empty

- [] Error

- [] Offline

- [] Permission denied

- [] Retry action

- [] Validation

- [] Success toast

Verification

- [] App builds

- [] Key routes render

- [] Forms validate

- [] API success path checked

- [] API failure path checked

- [] Responsive check completed

Notes for Tester

-

Final response format after implementation:

1. Implementation summary
2. Screens/routes implemented
3. API functions connected
4. Fallback states covered
5. Files changed
6. Verification performed with exact commands
7. Known backend gaps
8. Suggested commit message and PR title

V-- feat/payroll

AI Prompt

Prompt from product plan, shortened:

You are a senior frontend engineer building BillSync Legal, a React + Tailwind legal-tech SaaS platform. Use the Stitch UI design source at E:\Shwetal\UI Design\stitch_billsync_legal_platform_ui and the backend/API source at E:\Shwetal\legal-billing-backend. The implementation must cover the complete BillSync frontend scope: roughly 75 product screens plus 24 fallback, loading, empty, and error states. The frontend must connect to real backend APIs where they exist, create clean frontend service adapters where backend APIs are missing, and document every backend gap clearly. The UI is for lawyers and law firm operators who are not technical, so never expose raw technical words to users such as API, backend, payload, token, 401, CORS, endpoint, database, stack trace, or server exception. Translate all failures into calm, plain-language guidance. Use the BillSync palette: background #F8F5EF, panels #FFFFFF, navigation #0B1F3A, primary #123B63, accent gold #C9A227, text #1F2933, secondary text #6B7280, border #E5E7EB, success #15803D, warning #D97706, error #B91C1C. Use subtle purposeful animation only: login spinner, live work-meter clock, pulsing recording dot, extension connected check, recorder waveform, upload progress, AI shimmer, payment spinner, and sync progress.

Branch to create/work on:

feat/payroll

Screens/states/features to be added:

Payroll runs, payroll detail, compensation placeholders, payroll not configured state

Design reference:

Payroll screens

Backend/API reference:

Inspect E:\Shwetal\legal-billing-backend for route inventory, models, validators, controllers, tests, and docs related to this branch.

How to design:

If payroll APIs are missing, implement future-ready adapters and honest Phase/Not Configured UI.

API connection direction:

Create or update frontend service functions for each real backend route used by this branch. Normalize responses into UI-safe models. If a backend route is missing, add a clearly named adapter and document the backend gap in the final response and PR.

Branch-specific senior-engineer direction:

Favor simple, composable implementation over clever abstractions. Keep UI states explicit and reusable. Connect real backend data first; use honest not-configured states for unavailable MVP expansion APIs.

Acceptance criteria:

Payroll screens render without pretending backend exists; admin sees required setup steps.

Implementation guardrails:

- Start by inspecting relevant Stitch screens in the design folder.
- Start by inspecting relevant backend route files, controllers, validators, models, tests, and docs.
- Create or update API client functions before wiring pages.
- Connect every available screen action to backend behavior where available.

- Where backend APIs are missing, add a service adapter with a clear TODO/backend-gap note and show honest not-connected/not-configured UI.
- Include loading, empty, error, offline, permission, retry, success, and validation states for every data surface.
- Keep lawyer-facing copy non-technical, calm, and action-oriented.
- Do not modify backend unless the branch explicitly asks for a tiny compatibility fix.
- Verify desktop, tablet, and 360px mobile width.

Verification required before marking this branch complete:

- Run npm install only if dependencies are missing.
- Run npm run dev and manually verify the branch route(s).
- Run npm run build before final response.
- Check at least one API success path and one API failure/fallback path.
- Check at least one empty state.
- Check no horizontal overflow at 360px.
- Confirm user-facing copy contains no technical backend jargon.

Commit message:

feat(payroll): add payroll workflow shell

PR template before merging:

Summary

-

Screens / States Implemented

- []

API Mapping

- APIs connected:
 - `GET /api/...`
 - `POST /api/...`
- Frontend adapters added:
 - None / list adapters
- Backend gaps found:
 - None / list gaps

UX States Covered

- [] Loading
- [] Empty
- [] Error
- [] Offline
- [] Permission denied
- [] Retry action
- [] Validation
- [] Success toast

Verification

- [] App builds
- [] Key routes render
- [] Forms validate
- [] API success path checked
- [] API failure path checked
- [] Responsive check completed

Notes for Tester

-

Final response format after implementation:

1. Implementation summary
2. Screens/routes implemented
3. API functions connected
4. Fallback states covered
5. Files changed
6. Verification performed with exact commands
7. Known backend gaps
8. Suggested commit message and PR title

W-- feat/storage-documents

AI Prompt

Prompt from product plan, shortened:

You are a senior frontend engineer building BillSync Legal, a React + Tailwind legal-tech SaaS platform. Use the Stitch UI design source at E:\Shwetal\UI Design\stitch_billsync_legal_platform_ui and the backend/API source at E:\Shwetal\legal-billing-backend. The implementation must cover the complete BillSync frontend scope: roughly 75 product screens plus 24 fallback, loading, empty, and error states. The frontend must connect to real backend APIs where they exist, create clean frontend service adapters where backend APIs are missing, and document every backend gap clearly. The UI is for lawyers and law firm operators who are not technical, so never expose raw technical words to users such as API, backend, payload, token, 401, CORS, endpoint, database, stack trace, or server exception. Translate all failures into calm, plain-language guidance. Use the BillSync palette: background #F8F5EF, panels #FFFFFF, navigation #0B1F3A, primary #123B63, accent gold #C9A227, text #1F2933, secondary text #6B7280, border #E5E7EB, success #15803D, warning #D97706, error #B91C1C. Use subtle purposeful animation only: login spinner, live work-meter clock, pulsing recording dot, extension connected check, recorder waveform, upload progress, AI shimmer, payment spinner, and sync progress.

Branch to create/work on:

feat/storage-documents

Screens/states/features to be added:

Storage library, upload file, document viewer, storage settings, matter document links

Design reference:

Storage/document screens

Backend/API reference:

Inspect E:\Shwetal\legal-billing-backend for route inventory, models, validators, controllers, tests, and docs related to this branch.

How to design:

Wire Zoho WorkDrive/attachment APIs where available; create storage provider adapters for AWS/Google gaps.

API connection direction:

Create or update frontend service functions for each real backend route used by this branch. Normalize responses into UI-safe models. If a backend route is missing, add a clearly named adapter and document the backend gap in the final response and PR.

Branch-specific senior-engineer direction:

Favor simple, composable implementation over clever abstractions. Keep UI states explicit and reusable. Connect real backend data first; use honest not-configured states for unavailable MVP expansion APIs.

Acceptance criteria:

Document library/viewer/upload states render; provider-not-connected and upload-failed states exist.

Implementation guardrails:

- Start by inspecting relevant Stitch screens in the design folder.
- Start by inspecting relevant backend route files, controllers, validators, models, tests, and docs.
- Create or update API client functions before wiring pages.
- Connect every available screen action to backend behavior where available.
- Where backend APIs are missing, add a service adapter with a clear TODO/backend-gap note and show honest not-connected/not-configured UI.
- Include loading, empty, error, offline, permission, retry, success, and validation states for every data surface.
- Keep lawyer-facing copy non-technical, calm, and action-oriented.
- Do not modify backend unless the branch explicitly asks for a tiny compatibility fix.
- Verify desktop, tablet, and 360px mobile width.

Verification required before marking this branch complete:

- Run npm install only if dependencies are missing.
- Run npm run dev and manually verify the branch route(s).
- Run npm run build before final response.
- Check at least one API success path and one API failure/fallback path.
- Check at least one empty state.
- Check no horizontal overflow at 360px.
- Confirm user-facing copy contains no technical backend jargon.

Commit message:

feat(storage): connect document storage screens

PR template before merging:

Summary

-

Screens / States Implemented

- []

API Mapping

- APIs connected:

- `GET /api/...`

- `POST /api/...`

- Frontend adapters added:

- None / list adapters

- Backend gaps found:

- None / list gaps

UX States Covered

- [] Loading

- [] Empty

- [] Error

- [] Offline

- [] Permission denied

- [] Retry action

- [] Validation

- [] Success toast

Verification

- [] App builds

- [] Key routes render

- [] Forms validate

- [] API success path checked

- [] API failure path checked

- [] Responsive check completed

Notes for Tester

-

Final response format after implementation:

1. Implementation summary
2. Screens/routes implemented
3. API functions connected
4. Fallback states covered
5. Files changed
6. Verification performed with exact commands
7. Known backend gaps
8. Suggested commit message and PR title

X-- feat/ai-assistant-core

AI Prompt

Prompt from product plan, shortened:

You are a senior frontend engineer building BillSync Legal, a React + Tailwind legal-tech SaaS platform. Use the Stitch UI design source at E:\Shwetal\UI Design\stitch_billsync_legal_platform_ui and the backend/API source at E:\Shwetal\legal-billing-backend. The implementation must cover the complete BillSync frontend scope: roughly 75 product screens plus 24 fallback, loading, empty, and error states. The frontend must connect to real backend APIs where they exist, create clean frontend service adapters where backend APIs are missing, and document every backend gap clearly. The UI is for lawyers and law firm operators who are not technical, so never expose raw technical words to users such as API, backend, payload, token, 401, CORS, endpoint, database, stack trace, or server exception. Translate all failures into calm, plain-language guidance. Use the BillSync palette: background #F8F5EF, panels #FFFFFF, navigation #0B1F3A, primary #123B63, accent gold #C9A227, text #1F2933, secondary text #6B7280, border #E5E7EB, success #15803D, warning #D97706, error #B91C1C. Use subtle purposeful animation only: login spinner, live work-meter clock, pulsing recording dot, extension connected check, recorder waveform, upload progress, AI shimmer, payment spinner, and sync progress.

Branch to create/work on:

feat/ai-assistant-core

Screens/states/features to be added:

AI assistant, AI email writer, AI research assistant

Design reference:

AI assistant core screens

Backend/API reference:

Inspect E:\Shwetal\legal-billing-backend for route inventory, models, validators, controllers, tests, and docs related to this branch.

How to design:

Wire available AI endpoints for email and research/text assistance.

API connection direction:

Create or update frontend service functions for each real backend route used by this branch. Normalize responses into UI-safe models. If a backend route is missing, add a clearly named adapter and document the backend gap in the final response and PR.

Branch-specific senior-engineer direction:

Favor simple, composable implementation over clever abstractions. Keep UI states explicit and reusable. Connect real backend data first; use honest not-configured states for unavailable MVP expansion APIs.

Acceptance criteria:

AI modes call backend where available; delayed/unavailable states exist; outputs are clearly editable.

Implementation guardrails:

- Start by inspecting relevant Stitch screens in the design folder.
- Start by inspecting relevant backend route files, controllers, validators, models, tests, and docs.
- Create or update API client functions before wiring pages.
- Connect every available screen action to backend behavior where available.

- Where backend APIs are missing, add a service adapter with a clear TODO/backend-gap note and show honest not-connected/not-configured UI.
- Include loading, empty, error, offline, permission, retry, success, and validation states for every data surface.
- Keep lawyer-facing copy non-technical, calm, and action-oriented.
- Do not modify backend unless the branch explicitly asks for a tiny compatibility fix.
- Verify desktop, tablet, and 360px mobile width.

Verification required before marking this branch complete:

- Run npm install only if dependencies are missing.
- Run npm run dev and manually verify the branch route(s).
- Run npm run build before final response.
- Check at least one API success path and one API failure/fallback path.
- Check at least one empty state.
- Check no horizontal overflow at 360px.
- Confirm user-facing copy contains no technical backend jargon.

Commit message:

feat(ai): connect AI assistant core

PR template before merging:

Summary

-

Screens / States Implemented

- []

API Mapping

- APIs connected:
 - `GET /api/...`
 - `POST /api/...`
- Frontend adapters added:
 - None / list adapters
- Backend gaps found:
 - None / list gaps

UX States Covered

- [] Loading
- [] Empty
- [] Error
- [] Offline
- [] Permission denied
- [] Retry action
- [] Validation
- [] Success toast

Verification

- [] App builds
- [] Key routes render
- [] Forms validate
- [] API success path checked
- [] API failure path checked
- [] Responsive check completed

Notes for Tester

-

Final response format after implementation:

1. Implementation summary
2. Screens/routes implemented
3. API functions connected
4. Fallback states covered
5. Files changed
6. Verification performed with exact commands
7. Known backend gaps
8. Suggested commit message and PR title

Y-- feat/ai-documents-rag

AI Prompt

Prompt from product plan, shortened:

You are a senior frontend engineer building BillSync Legal, a React + Tailwind legal-tech SaaS platform. Use the Stitch UI design source at E:\Shwetal\UI Design\stitch_billsync_legal_platform_ui and the backend/API source at E:\Shwetal\legal-billing-backend. The implementation must cover the complete BillSync frontend scope: roughly 75 product screens plus 24 fallback, loading, empty, and error states. The frontend must connect to real backend APIs where they exist, create clean frontend service adapters where backend APIs are missing, and document every backend gap clearly. The UI is for lawyers and law firm operators who are not technical, so never expose raw technical words to users such as API, backend, payload, token, 401, CORS, endpoint, database, stack trace, or server exception. Translate all failures into calm, plain-language guidance. Use the BillSync palette: background #F8F5EF, panels #FFFFFF, navigation #0B1F3A, primary #123B63, accent gold #C9A227, text #1F2933, secondary text #6B7280, border #E5E7EB, success #15803D, warning #D97706, error #B91C1C. Use subtle purposeful animation only: login spinner, live work-meter clock, pulsing recording dot, extension connected check, recorder waveform, upload progress, AI shimmer, payment spinner, and sync progress.

Branch to create/work on:

feat/ai-documents-rag

Screens/states/features to be added:

Document summary, document creation, RAG chatbot, matter document Q&A shell

Design reference:

AI document/RAG screens

Backend/API reference:

Inspect E:\Shwetal\legal-billing-backend for route inventory, models, validators, controllers, tests, and docs related to this branch.

How to design:

Wire available AI/document APIs. If RAG indexing is missing, show indexing-required state and adapter TODO.

API connection direction:

Create or update frontend service functions for each real backend route used by this branch. Normalize responses into UI-safe models. If a backend route is missing, add a clearly named adapter and document the backend gap in the final response and PR.

Branch-specific senior-engineer direction:

Favor simple, composable implementation over clever abstractions. Keep UI states explicit and reusable. Connect real backend data first; use honest not-configured states for unavailable MVP expansion APIs.

Acceptance criteria:

Doc summary/create shells render; RAG does not fake document-grounded answers without backend support.

Implementation guardrails:

- Start by inspecting relevant Stitch screens in the design folder.
- Start by inspecting relevant backend route files, controllers, validators, models, tests, and docs.
- Create or update API client functions before wiring pages.
- Connect every available screen action to backend behavior where available.
- Where backend APIs are missing, add a service adapter with a clear TODO/backend-gap note and show honest not-connected/not-configured UI.
- Include loading, empty, error, offline, permission, retry, success, and validation states for every data surface.
- Keep lawyer-facing copy non-technical, calm, and action-oriented.
- Do not modify backend unless the branch explicitly asks for a tiny compatibility fix.
- Verify desktop, tablet, and 360px mobile width.

Verification required before marking this branch complete:

- Run npm install only if dependencies are missing.
- Run npm run dev and manually verify the branch route(s).
- Run npm run build before final response.
- Check at least one API success path and one API failure/fallback path.
- Check at least one empty state.
- Check no horizontal overflow at 360px.
- Confirm user-facing copy contains no technical backend jargon.

Commit message:

feat(ai): add document summary creation and RAG screens

PR template before merging:

Summary

-

Screens / States Implemented

- []

API Mapping

- APIs connected:

- `GET /api/...`

- `POST /api/...`

- Frontend adapters added:

- None / list adapters

- Backend gaps found:

- None / list gaps

UX States Covered

- [] Loading

- [] Empty

- [] Error

- [] Offline

- [] Permission denied

- [] Retry action

- [] Validation

- [] Success toast

Verification

- [] App builds

- [] Key routes render

- [] Forms validate

- [] API success path checked

- [] API failure path checked

- [] Responsive check completed

Notes for Tester

-

Final response format after implementation:

1. Implementation summary
2. Screens/routes implemented
3. API functions connected
4. Fallback states covered
5. Files changed
6. Verification performed with exact commands
7. Known backend gaps
8. Suggested commit message and PR title

Z-- feat/communications-whatsapp-sms

AI Prompt

Prompt from product plan, shortened:

You are a senior frontend engineer building BillSync Legal, a React + Tailwind legal-tech SaaS platform. Use the Stitch UI design source at E:\Shwetal\UI Design\stitch_billsync_legal_platform_ui and the backend/API source at E:\Shwetal\legal-billing-backend. The implementation must cover the complete BillSync frontend scope: roughly 75 product screens plus 24 fallback, loading, empty, and error states. The frontend must connect to real backend APIs where they exist, create clean frontend service adapters where backend APIs are missing, and document every backend gap clearly. The UI is for lawyers and law firm operators who are not technical, so never expose raw technical words to users such as API, backend, payload, token, 401, CORS, endpoint, database, stack trace, or server exception. Translate all failures into calm, plain-language guidance. Use the BillSync palette: background #F8F5EF, panels #FFFFFF, navigation #0B1F3A, primary #123B63, accent gold #C9A227, text #1F2933, secondary text #6B7280, border #E5E7EB, success #15803D, warning #D97706, error #B91C1C. Use subtle purposeful animation only: login spinner, live work-meter clock, pulsing recording dot, extension connected check, recorder waveform, upload progress, AI shimmer, payment spinner, and sync progress.

Branch to create/work on:

feat/communications-whatsapp-sms

Screens/states/features to be added:

WhatsApp inbox/templates, SMS center/templates, communication logs

Design reference:

WhatsApp/SMS screens

Backend/API reference:

Inspect E:\Shwetal\legal-billing-backend for route inventory, models, validators, controllers, tests, and docs related to this branch.

How to design:

If WhatsApp/SMS APIs are missing, add provider setup, not connected, and logs adapters.

API connection direction:

Create or update frontend service functions for each real backend route used by this branch. Normalize responses into UI-safe models. If a backend route is missing, add a clearly named adapter and document the backend gap in the final response and PR.

Branch-specific senior-engineer direction:

Favor simple, composable implementation over clever abstractions. Keep UI states explicit and reusable. Connect real backend data first; use honest not-configured states for unavailable MVP expansion APIs.

Acceptance criteria:

Communication screens render; no fake send confirmation; provider status is clear.

Implementation guardrails:

- Start by inspecting relevant Stitch screens in the design folder.
- Start by inspecting relevant backend route files, controllers, validators, models, tests, and docs.
- Create or update API client functions before wiring pages.
- Connect every available screen action to backend behavior where available.

- Where backend APIs are missing, add a service adapter with a clear TODO/backend-gap note and show honest not-connected/not-configured UI.
- Include loading, empty, error, offline, permission, retry, success, and validation states for every data surface.
- Keep lawyer-facing copy non-technical, calm, and action-oriented.
- Do not modify backend unless the branch explicitly asks for a tiny compatibility fix.
- Verify desktop, tablet, and 360px mobile width.

Verification required before marking this branch complete:

- Run npm install only if dependencies are missing.
- Run npm run dev and manually verify the branch route(s).
- Run npm run build before final response.
- Check at least one API success path and one API failure/fallback path.
- Check at least one empty state.
- Check no horizontal overflow at 360px.
- Confirm user-facing copy contains no technical backend jargon.

Commit message:

feat(comms): add WhatsApp SMS communication screens

PR template before merging:

Summary

-

Screens / States Implemented

- []

API Mapping

- APIs connected:
 - `GET /api/...`
 - `POST /api/...`
- Frontend adapters added:
 - None / list adapters
- Backend gaps found:
 - None / list gaps

UX States Covered

- [] Loading
- [] Empty
- [] Error
- [] Offline
- [] Permission denied
- [] Retry action
- [] Validation
- [] Success toast

Verification

- [] App builds
- [] Key routes render
- [] Forms validate
- [] API success path checked
- [] API failure path checked
- [] Responsive check completed

Notes for Tester

-

Final response format after implementation:

1. Implementation summary
2. Screens/routes implemented
3. API functions connected
4. Fallback states covered
5. Files changed
6. Verification performed with exact commands
7. Known backend gaps
8. Suggested commit message and PR title

AA-- feat/court-daily-sync

AI Prompt

Prompt from product plan, shortened:

You are a senior frontend engineer building BillSync Legal, a React + Tailwind legal-tech SaaS platform. Use the Stitch UI design source at E:\Shweta\UI Design\stitch_billsync_legal_platform_ui and the backend/API source at E:\Shweta\legal-billing-backend. The implementation must cover the complete BillSync frontend scope: roughly 75 product screens plus 24 fallback, loading, empty, and error states. The frontend must connect to real backend APIs where they exist, create clean frontend service adapters where backend APIs are missing, and document every backend gap clearly. The UI is for lawyers and law firm operators who are not technical, so never expose raw technical words to users such as API, backend, payload, token, 401, CORS, endpoint, database, stack trace, or server exception. Translate all failures into calm, plain-language guidance. Use the BillSync palette: background #F8F5EF, panels #FFFFFF, navigation #0B1F3A, primary #123B63, accent gold #C9A227, text #1F2933, secondary text #6B7280, border #E5E7EB, success #15803D, warning #D97706, error #B91C1C. Use subtle purposeful animation only: login spinner, live work-meter clock, pulsing recording dot, extension connected check, recorder waveform, upload progress, AI shimmer, payment spinner, and sync progress.

Branch to create/work on:

feat/court-daily-sync

Screens/states/features to be added:

Court daily sync dashboard, court case match, verdict detail, court sync settings

Design reference:

Court sync screens

Backend/API reference:

Inspect E:\Shwetal\legal-billing-backend for route inventory, models, validators, controllers, tests, and docs related to this branch.

How to design:

If court feed APIs are missing, build not-connected/sync-failed states and matter-linking UI shell.

API connection direction:

Create or update frontend service functions for each real backend route used by this branch. Normalize responses into UI-safe models. If a backend route is missing, add a clearly named adapter and document the backend gap in the final response and PR.

Branch-specific senior-engineer direction:

Favor simple, composable implementation over clever abstractions. Keep UI states explicit and reusable. Connect real backend data first; use honest not-configured states for unavailable MVP expansion APIs.

Acceptance criteria:

Court screens render; manual hearing and live court sync are visually distinct.

Implementation guardrails:

- Start by inspecting relevant Stitch screens in the design folder.
- Start by inspecting relevant backend route files, controllers, validators, models, tests, and docs.
- Create or update API client functions before wiring pages.
- Connect every available screen action to backend behavior where available.
- Where backend APIs are missing, add a service adapter with a clear TODO/backend-gap note and show honest not-connected/not-configured UI.
- Include loading, empty, error, offline, permission, retry, success, and validation states for every data surface.
- Keep lawyer-facing copy non-technical, calm, and action-oriented.
- Do not modify backend unless the branch explicitly asks for a tiny compatibility fix.
- Verify desktop, tablet, and 360px mobile width.

Verification required before marking this branch complete:

- Run npm install only if dependencies are missing.
- Run npm run dev and manually verify the branch route(s).
- Run npm run build before final response.
- Check at least one API success path and one API failure/fallback path.
- Check at least one empty state.
- Check no horizontal overflow at 360px.
- Confirm user-facing copy contains no technical backend jargon.

Commit message:

feat(court): add court daily sync screens

PR template before merging:

Summary

-

Screens / States Implemented

- []

API Mapping

- APIs connected:

- `GET /api/...`

- `POST /api/...`

- Frontend adapters added:

- None / list adapters

- Backend gaps found:

- None / list gaps

UX States Covered

- [] Loading

- [] Empty

- [] Error

- [] Offline

- [] Permission denied

- [] Retry action

- [] Validation

- [] Success toast

Verification

- [] App builds

- [] Key routes render

- [] Forms validate

- [] API success path checked

- [] API failure path checked

- [] Responsive check completed

Notes for Tester

-

Final response format after implementation:

1. Implementation summary
2. Screens/routes implemented
3. API functions connected
4. Fallback states covered
5. Files changed
6. Verification performed with exact commands
7. Known backend gaps
8. Suggested commit message and PR title

AB-- feat/zoho-integration-api

AI Prompt

Prompt from product plan, shortened:

You are a senior frontend engineer building BillSync Legal, a React + Tailwind legal-tech SaaS platform. Use the Stitch UI design source at E:\Shwetal\UI Design\stitch_billsync_legal_platform_ui and the backend/API source at E:\Shwetal\legal-billing-backend. The implementation must cover the complete BillSync frontend scope: roughly 75 product screens plus 24 fallback, loading, empty, and error states. The frontend must connect to real backend APIs where they exist, create clean frontend service adapters where backend APIs are missing, and document every backend gap clearly. The UI is for lawyers and law firm operators who are not technical, so never expose raw technical words to users such as API, backend, payload, token, 401, CORS, endpoint, database, stack trace, or server exception. Translate all failures into calm, plain-language guidance. Use the BillSync palette: background #F8F5EF, panels #FFFFFF, navigation #0B1F3A, primary #123B63, accent gold #C9A227, text #1F2933, secondary text #6B7280, border #E5E7EB, success #15803D, warning #D97706, error #B91C1C. Use subtle purposeful animation only: login spinner, live work-meter clock, pulsing recording dot, extension connected check, recorder waveform, upload progress, AI shimmer, payment spinner, and sync progress.

Branch to create/work on:

feat/zoho-integration-api

Screens/states/features to be added:

Zoho connect/status/sync, WorkDrive link, Zoho sync logs

Design reference:

Zoho integration screens

Backend/API reference:

Inspect E:\Shwetal\legal-billing-backend for route inventory, models, validators, controllers, tests, and docs related to this branch.

How to design:

Wire backend Zoho auth/status/sync/workdrive/attachment routes.

API connection direction:

Create or update frontend service functions for each real backend route used by this branch. Normalize responses into UI-safe models. If a backend route is missing, add a clearly named adapter and document the backend gap in the final response and PR.

Branch-specific senior-engineer direction:

Favor simple, composable implementation over clever abstractions. Keep UI states explicit and reusable. Connect real backend data first; use honest not-configured states for unavailable MVP expansion APIs.

Acceptance criteria:

Zoho connect/status works; sync actions and logs render; provider errors are friendly.

Implementation guardrails:

- Start by inspecting relevant Stitch screens in the design folder.
- Start by inspecting relevant backend route files, controllers, validators, models, tests, and docs.
- Create or update API client functions before wiring pages.
- Connect every available screen action to backend behavior where available.

- Where backend APIs are missing, add a service adapter with a clear TODO/backend-gap note and show honest not-connected/not-configured UI.
- Include loading, empty, error, offline, permission, retry, success, and validation states for every data surface.
- Keep lawyer-facing copy non-technical, calm, and action-oriented.
- Do not modify backend unless the branch explicitly asks for a tiny compatibility fix.
- Verify desktop, tablet, and 360px mobile width.

Verification required before marking this branch complete:

- Run npm install only if dependencies are missing.
- Run npm run dev and manually verify the branch route(s).
- Run npm run build before final response.
- Check at least one API success path and one API failure/fallback path.
- Check at least one empty state.
- Check no horizontal overflow at 360px.
- Confirm user-facing copy contains no technical backend jargon.

Commit message:

feat(integrations): connect Zoho workflows

PR template before merging:

Summary

-

Screens / States Implemented

- []

API Mapping

- APIs connected:
 - `GET /api/...`
 - `POST /api/...`
- Frontend adapters added:
 - None / list adapters
- Backend gaps found:
 - None / list gaps

UX States Covered

- [] Loading
- [] Empty
- [] Error
- [] Offline
- [] Permission denied
- [] Retry action
- [] Validation
- [] Success toast

Verification

- [] App builds
- [] Key routes render
- [] Forms validate
- [] API success path checked
- [] API failure path checked
- [] Responsive check completed

Notes for Tester

-

Final response format after implementation:

1. Implementation summary
2. Screens/routes implemented
3. API functions connected
4. Fallback states covered
5. Files changed
6. Verification performed with exact commands
7. Known backend gaps
8. Suggested commit message and PR title

AC-- feat/aws-google-integrations

AI Prompt

Prompt from product plan, shortened:

You are a senior frontend engineer building BillSync Legal, a React + Tailwind legal-tech SaaS platform. Use the Stitch UI design source at E:\Shwetal\UI Design\stitch_billsync_legal_platform_ui and the backend/API source at E:\Shwetal\legal-billing-backend. The implementation must cover the complete BillSync frontend scope: roughly 75 product screens plus 24 fallback, loading, empty, and error states. The frontend must connect to real backend APIs where they exist, create clean frontend service adapters where backend APIs are missing, and document every backend gap clearly. The UI is for lawyers and law firm operators who are not technical, so never expose raw technical words to users such as API, backend, payload, token, 401, CORS, endpoint, database, stack trace, or server exception. Translate all failures into calm, plain-language guidance. Use the BillSync palette: background #F8F5EF, panels #FFFFFF, navigation #0B1F3A, primary #123B63, accent gold #C9A227, text #1F2933, secondary text #6B7280, border #E5E7EB, success #15803D, warning #D97706, error #B91C1C. Use subtle purposeful animation only: login spinner, live work-meter clock, pulsing recording dot, extension connected check, recorder waveform, upload progress, AI shimmer, payment spinner, and sync progress.

Branch to create/work on:

feat/aws-google-integrations

Screens/states/features to be added:

AWS/Google integration setup, cloud storage provider status, integration not connected states

Design reference:

AWS/Google integration screens

Backend/API reference:

Inspect E:\Shwetal\legal-billing-backend for route inventory, models, validators, controllers, tests, and docs related to this branch.

How to design:

If APIs are missing, create provider adapters and honest setup requirements.

API connection direction:

Create or update frontend service functions for each real backend route used by this branch. Normalize responses into UI-safe models. If a backend route is missing, add a clearly named adapter and document the backend gap in the final response and PR.

Branch-specific senior-engineer direction:

Favor simple, composable implementation over clever abstractions. Keep UI states explicit and reusable. Connect real backend data first; use honest not-configured states for unavailable MVP expansion APIs.

Acceptance criteria:

AWS/Google screens render; not-connected and setup states are clear.

Implementation guardrails:

- Start by inspecting relevant Stitch screens in the design folder.
- Start by inspecting relevant backend route files, controllers, validators, models, tests, and docs.
- Create or update API client functions before wiring pages.
- Connect every available screen action to backend behavior where available.
- Where backend APIs are missing, add a service adapter with a clear TODO/backend-gap note and show honest not-connected/not-configured UI.
- Include loading, empty, error, offline, permission, retry, success, and validation states for every data surface.
- Keep lawyer-facing copy non-technical, calm, and action-oriented.
- Do not modify backend unless the branch explicitly asks for a tiny compatibility fix.
- Verify desktop, tablet, and 360px mobile width.

Verification required before marking this branch complete:

- Run npm install only if dependencies are missing.
- Run npm run dev and manually verify the branch route(s).
- Run npm run build before final response.
- Check at least one API success path and one API failure/fallback path.
- Check at least one empty state.
- Check no horizontal overflow at 360px.
- Confirm user-facing copy contains no technical backend jargon.

Commit message:

feat(integrations): add AWS and Google provider screens

PR template before merging:

Summary

-

Screens / States Implemented

- []

API Mapping

- APIs connected:

- `GET /api/...`

- `POST /api/...`

- Frontend adapters added:

- None / list adapters

- Backend gaps found:

- None / list gaps

UX States Covered

- [] Loading

- [] Empty

- [] Error

- [] Offline

- [] Permission denied

- [] Retry action

- [] Validation

- [] Success toast

Verification

- [] App builds

- [] Key routes render

- [] Forms validate

- [] API success path checked

- [] API failure path checked

- [] Responsive check completed

Notes for Tester

-

Final response format after implementation:

1. Implementation summary
2. Screens/routes implemented
3. API functions connected
4. Fallback states covered
5. Files changed
6. Verification performed with exact commands
7. Known backend gaps
8. Suggested commit message and PR title

AD-- feat/settings-admin-controls

AI Prompt

Prompt from product plan, shortened:

You are a senior frontend engineer building BillSync Legal, a React + Tailwind legal-tech SaaS platform. Use the Stitch UI design source at E:\Shweta\UI Design\stitch_billsync_legal_platform_ui and the backend/API source at E:\Shweta\legal-billing-backend. The implementation must cover the complete BillSync frontend scope: roughly 75 product screens plus 24 fallback, loading, empty, and error states. The frontend must connect to real backend APIs where they exist, create clean frontend service adapters where backend APIs are missing, and document every backend gap clearly. The UI is for lawyers and law firm operators who are not technical, so never expose raw technical words to users such as API, backend, payload, token, 401, CORS, endpoint, database, stack trace, or server exception. Translate all failures into calm, plain-language guidance. Use the BillSync palette: background #F8F5EF, panels #FFFFFF, navigation #0B1F3A, primary #123B63, accent gold #C9A227, text #1F2933, secondary text #6B7280, border #E5E7EB, success #15803D, warning #D97706, error #B91C1C. Use subtle purposeful animation only: login spinner, live work-meter clock, pulsing recording dot, extension connected check, recorder waveform, upload progress, AI shimmer, payment spinner, and sync progress.

Branch to create/work on:

feat/settings-admin-controls

Screens/states/features to be added:

Settings, firm setup, permissions matrix refinements, invoice/tax/storage/notification defaults

Design reference:

Settings/admin screens

Backend/API reference:

Inspect E:\Shweta\legal-billing-backend for route inventory, models, validators, controllers, tests, and docs related to this branch.

How to design:

Wire available firm/user/settings APIs and keep unavailable settings visibly disabled or not configured.

API connection direction:

Create or update frontend service functions for each real backend route used by this branch. Normalize responses into UI-safe models. If a backend route is missing, add a clearly named adapter and document the backend gap in the final response and PR.

Branch-specific senior-engineer direction:

Favor simple, composable implementation over clever abstractions. Keep UI states explicit and reusable. Connect real backend data first; use honest not-configured states for unavailable MVP expansion APIs.

Acceptance criteria:

Settings screens render; admin-only gating works; validation/fallback states exist.

Implementation guardrails:

- Start by inspecting relevant Stitch screens in the design folder.
- Start by inspecting relevant backend route files, controllers, validators, models, tests, and docs.
- Create or update API client functions before wiring pages.
- Connect every available screen action to backend behavior where available.

- Where backend APIs are missing, add a service adapter with a clear TODO/backend-gap note and show honest not-connected/not-configured UI.
- Include loading, empty, error, offline, permission, retry, success, and validation states for every data surface.
- Keep lawyer-facing copy non-technical, calm, and action-oriented.
- Do not modify backend unless the branch explicitly asks for a tiny compatibility fix.
- Verify desktop, tablet, and 360px mobile width.

Verification required before marking this branch complete:

- Run npm install only if dependencies are missing.
- Run npm run dev and manually verify the branch route(s).
- Run npm run build before final response.
- Check at least one API success path and one API failure/fallback path.
- Check at least one empty state.
- Check no horizontal overflow at 360px.
- Confirm user-facing copy contains no technical backend jargon.

Commit message:

feat(settings): add admin settings and controls

PR template before merging:

Summary

-

Screens / States Implemented

- []

API Mapping

- APIs connected:
 - `GET /api/...`
 - `POST /api/...`
- Frontend adapters added:
 - None / list adapters
- Backend gaps found:
 - None / list gaps

UX States Covered

- [] Loading
- [] Empty
- [] Error
- [] Offline
- [] Permission denied
- [] Retry action
- [] Validation
- [] Success toast

Verification

- [] App builds
- [] Key routes render
- [] Forms validate
- [] API success path checked
- [] API failure path checked
- [] Responsive check completed

Notes for Tester

-

Final response format after implementation:

1. Implementation summary
2. Screens/routes implemented
3. API functions connected
4. Fallback states covered
5. Files changed
6. Verification performed with exact commands
7. Known backend gaps
8. Suggested commit message and PR title

AE-- style/responsive-accessibility-copy-polish

AI Prompt

Prompt from product plan, shortened:

You are a senior frontend engineer building BillSync Legal, a React + Tailwind legal-tech SaaS platform. Use the Stitch UI design source at E:\Shwetal\UI Design\stitch_billsync_legal_platform_ui and the backend/API source at E:\Shwetal\legal-billing-backend. The implementation must cover the complete BillSync frontend scope: roughly 75 product screens plus 24 fallback, loading, empty, and error states. The frontend must connect to real backend APIs where they exist, create clean frontend service adapters where backend APIs are missing, and document every backend gap clearly. The UI is for lawyers and law firm operators who are not technical, so never expose raw technical words to users such as API, backend, payload, token, 401, CORS, endpoint, database, stack trace, or server exception. Translate all failures into calm, plain-language guidance. Use the BillSync palette: background #F8F5EF, panels #FFFFFF, navigation #0B1F3A, primary #123B63, accent gold #C9A227, text #1F2933, secondary text #6B7280, border #E5E7EB, success #15803D, warning #D97706, error #B91C1C. Use subtle purposeful animation only: login spinner, live work-meter clock, pulsing recording dot, extension connected check, recorder waveform, upload progress, AI shimmer, payment spinner, and sync progress.

Branch to create/work on:

style/responsive-accessibility-copy-polish

Screens/states/features to be added:

Responsive polish, accessibility, keyboard/focus, contrast, text overflow, reduced motion, lawyer-friendly copy pass

Design reference:

All screens

Backend/API reference:

Inspect E:\Shweta\legal-billing-backend for route inventory, models, validators, controllers, tests, and docs related to this branch.

How to design:

Audit every route and state after implementation. Fix cramped layouts, overflow, weak contrast, and technical copy.

API connection direction:

Create or update frontend service functions for each real backend route used by this branch. Normalize responses into UI-safe models. If a backend route is missing, add a clearly named adapter and document the backend gap in the final response and PR.

Branch-specific senior-engineer direction:

Favor simple, composable implementation over clever abstractions. Keep UI states explicit and reusable. Connect real backend data first; use honest not-configured states for unavailable MVP expansion APIs.

Acceptance criteria:

No horizontal overflow at 360px; focus states visible; reduced-motion respected; copy is non-technical.

Implementation guardrails:

- Start by inspecting relevant Stitch screens in the design folder.
- Start by inspecting relevant backend route files, controllers, validators, models, tests, and docs.
- Create or update API client functions before wiring pages.
- Connect every available screen action to backend behavior where available.
- Where backend APIs are missing, add a service adapter with a clear TODO/backend-gap note and show honest not-connected/not-configured UI.
- Include loading, empty, error, offline, permission, retry, success, and validation states for every data surface.
- Keep lawyer-facing copy non-technical, calm, and action-oriented.
- Do not modify backend unless the branch explicitly asks for a tiny compatibility fix.
- Verify desktop, tablet, and 360px mobile width.

Verification required before marking this branch complete:

- Run npm install only if dependencies are missing.
- Run npm run dev and manually verify the branch route(s).
- Run npm run build before final response.
- Check at least one API success path and one API failure/fallback path.
- Check at least one empty state.
- Check no horizontal overflow at 360px.
- Confirm user-facing copy contains no technical backend jargon.

Commit message:

style(ui): polish responsiveness accessibility and copy

PR template before merging:

Summary

-

Screens / States Implemented

- []

API Mapping

- APIs connected:

- `GET /api/...`

- `POST /api/...`

- Frontend adapters added:

- None / list adapters

- Backend gaps found:

- None / list gaps

UX States Covered

- [] Loading

- [] Empty

- [] Error

- [] Offline

- [] Permission denied

- [] Retry action

- [] Validation

- [] Success toast

Verification

- [] App builds

- [] Key routes render

- [] Forms validate

- [] API success path checked

- [] API failure path checked

- [] Responsive check completed

Notes for Tester

-

Final response format after implementation:

1. Implementation summary
2. Screens/routes implemented
3. API functions connected
4. Fallback states covered
5. Files changed
6. Verification performed with exact commands
7. Known backend gaps
8. Suggested commit message and PR title

AF-- test/e2e-demo-hardening

AI Prompt

Prompt from product plan, shortened:

You are a senior frontend engineer building BillSync Legal, a React + Tailwind legal-tech SaaS platform. Use the Stitch UI design source at E:\Shwetal\UI Design\stitch_billsync_legal_platform_ui and the backend/API source at E:\Shwetal\legal-billing-backend. The implementation must cover the complete BillSync frontend scope: roughly 75 product screens plus 24 fallback, loading, empty, and error states. The frontend must connect to real backend APIs where they exist, create clean frontend service adapters where backend APIs are missing, and document every backend gap clearly. The UI is for lawyers and law firm operators who are not technical, so never expose raw technical words to users such as API, backend, payload, token, 401, CORS, endpoint, database, stack trace, or server exception. Translate all failures into calm, plain-language guidance. Use the BillSync palette: background #F8F5EF, panels #FFFFFF, navigation #0B1F3A, primary #123B63, accent gold #C9A227, text #1F2933, secondary text #6B7280, border #E5E7EB, success #15803D, warning #D97706, error #B91C1C. Use subtle purposeful animation only: login spinner, live work-meter clock, pulsing recording dot, extension connected check, recorder waveform, upload progress, AI shimmer, payment spinner, and sync progress.

Branch to create/work on:

test/e2e-demo-hardening

Screens/states/features to be added:

End-to-end demo QA across all product screens and fallback states, smoke tests, seeded-data assumptions, final backend-gap report

Design reference:

All screens and testing path

Backend/API reference:

Inspect E:\Shwetal\legal-billing-backend for route inventory, models, validators, controllers, tests, and docs related to this branch.

How to design:

Run the complete product path for lawyer/admin/finance roles and harden anything blocking testing.

API connection direction:

Create or update frontend service functions for each real backend route used by this branch. Normalize responses into UI-safe models. If a backend route is missing, add a clearly named adapter and document the backend gap in the final response and PR.

Branch-specific senior-engineer direction:

Favor simple, composable implementation over clever abstractions. Keep UI states explicit and reusable. Connect real backend data first; use honest not-configured states for unavailable MVP expansion APIs.

Acceptance criteria:

Build passes; demo path works; backend gaps are documented; all major fallbacks are reachable.

Implementation guardrails:

- Start by inspecting relevant Stitch screens in the design folder.
- Start by inspecting relevant backend route files, controllers, validators, models, tests, and docs.
- Create or update API client functions before wiring pages.

- Connect every available screen action to backend behavior where available.
- Where backend APIs are missing, add a service adapter with a clear TODO/backend-gap note and show honest not-connected/not-configured UI.
- Include loading, empty, error, offline, permission, retry, success, and validation states for every data surface.
- Keep lawyer-facing copy non-technical, calm, and action-oriented.
- Do not modify backend unless the branch explicitly asks for a tiny compatibility fix.
- Verify desktop, tablet, and 360px mobile width.

Verification required before marking this branch complete:

- Run npm install only if dependencies are missing.
- Run npm run dev and manually verify the branch route(s).
- Run npm run build before final response.
- Check at least one API success path and one API failure/fallback path.
- Check at least one empty state.
- Check no horizontal overflow at 360px.
- Confirm user-facing copy contains no technical backend jargon.

Commit message:

test(frontend): harden full product demo flows

PR template before merging:

Summary

-

Screens / States Implemented

- []

API Mapping

- APIs connected:
 - `GET /api/...`
 - `POST /api/...`
- Frontend adapters added:
 - None / list adapters
- Backend gaps found:
 - None / list gaps

UX States Covered

- [] Loading
- [] Empty
- [] Error
- [] Offline
- [] Permission denied
- [] Retry action
- [] Validation
- [] Success toast

Verification

- [] App builds
- [] Key routes render
- [] Forms validate
- [] API success path checked
- [] API failure path checked
- [] Responsive check completed

Notes for Tester

-

Final response format after implementation:

1. Implementation summary
2. Screens/routes implemented
3. API functions connected
4. Fallback states covered
5. Files changed
6. Verification performed with exact commands
7. Known backend gaps
8. Suggested commit message and PR title

Universal Merge Rule

Merge only when:

- The branch builds.
- Routes render on desktop and mobile.
- API success and failure states exist.
- User-facing copy has no technical jargon.
- Screens match the Stitch visual direction.
- Backend gaps are documented, not hidden.
- PR template is completed before merging.